

EXP NO.1a	<u>VIRTUAL LABS</u> PROGRAMMING FOR DELAY GENERATION AND EFFECT OF CPU CLOCK
DATE: 03.07.26	

AIM:

To study the effect of CPU clock on Microcontrollers Operations.

PROCEDURE:**Simulator :**

- step 1 : Select clock frequency.
- step 2 : Enter Total Delay(ms) between 1 ms to 1000 ms.
- step 3 : Enter ON Delay and OFF Delay in milliseconds. The Addition of ON Delay and OFF Delay must be same as Total Delay.
- step 4 : Click on "SUBMIT" button.
- step 5 : Select Pin.
- step 6 : Select LED Configuration.
- step 7 : Click on "GENERATE CODE" button to generate code.
- step 8 : Click on "COMPILE" button to compile code.
- step 9 : Click on "EXECUTE" button to execute code and goto mimic.
- step 10 : If you want to download code click on "DOWNLOAD" button.
- step 11 : The MIMIC will open on left side of screen.
- step 12 : Click on selected pin terminal(red circle) and click on anode terminal(red circle) for connection of the selected pin to anode or vice versa.
- step 13 : Click on cathode terminal(red circle) and click on resistor terminal(red circle) for connection of cathode to resistor or vice versa.
- step 14 : Click on "Check Connection" button.
- step 15 : If connection is wrong, click on "Reset Connection" button and follow step 1, 2 and 3.
- step 16 : If connection is successful, click on "Run" button.
- step 17 : Output LED is on. Observe the square wave graph.
- step 18 : Click on "REFRESH" button to re-executing the experiment.

SIMULATION:**INPUT CONFIGURATION 1:**

The generated waveform is a square wave with a total period of 1000 ms. The output remains HIGH (LED ON) for 700 ms and LOW (LED OFF) for 300 ms in each cycle. Since the ON delay is greater than the OFF delay, the HIGH pulse is wider than the LOW pulse, resulting in the LED staying ON for a longer duration than it stays OFF.

CONFIGURATION

Select Clock Frequency: 24 MHz

Enter Total Delay(ms): 1000

Enter ON-OFF Delay(ms): 700 300

SUBMIT

Select Pin: RD3

Select LED Configuration: Current Sourcing

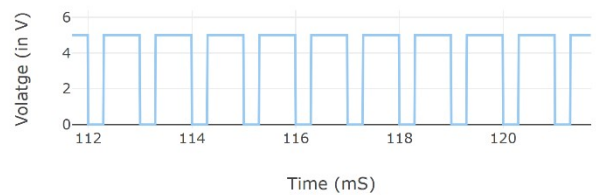
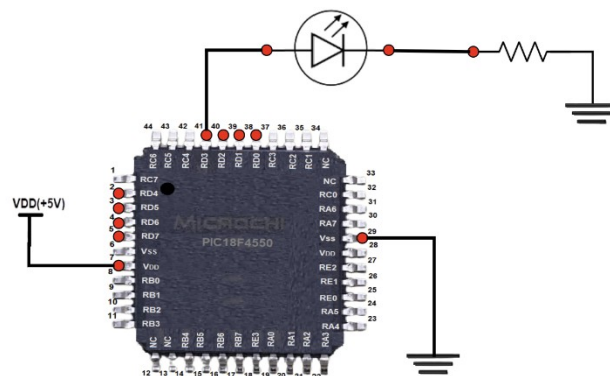
OUTPUT 1:

Check Connection

Run

Reset Connection

Delay Generation Graph



INPUT CONFIGURATION 2:

The generated waveform is a square wave with a total period of 800 ms. The output remains HIGH (LED ON) for 300 ms and LOW (LED OFF) for 500 ms in each cycle. Since the OFF delay is greater than the ON delay, the LOW pulse is wider than the HIGH pulse, resulting in the LED staying OFF for a longer duration than it stays ON.

CONFIGURATION

Select Clock Frequency: 16 MHz

Enter Total Delay(ms): 800

Enter ON-OFF Delay(ms): 300 500

SUBMIT

Select Pin: RD4

Select LED Configuration: Current Sourcing

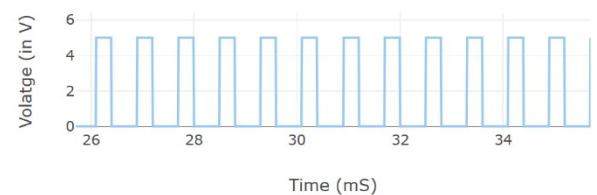
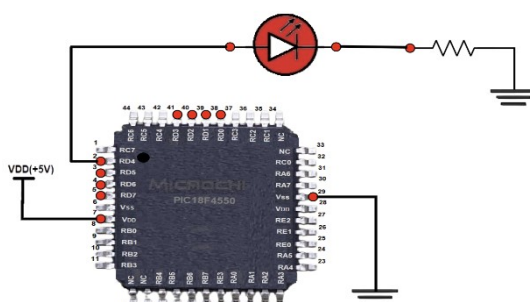
OUTPUT 2:

Check Connection

Run

Reset Connection

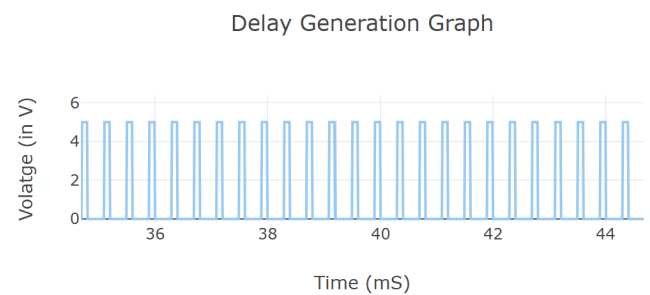
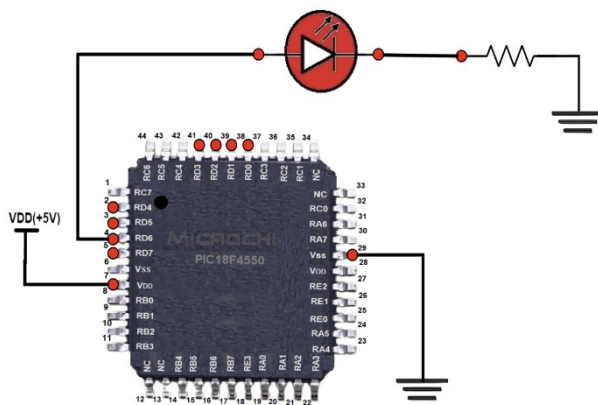
Delay Generation Graph



INPUT CONFIGURATION 3:

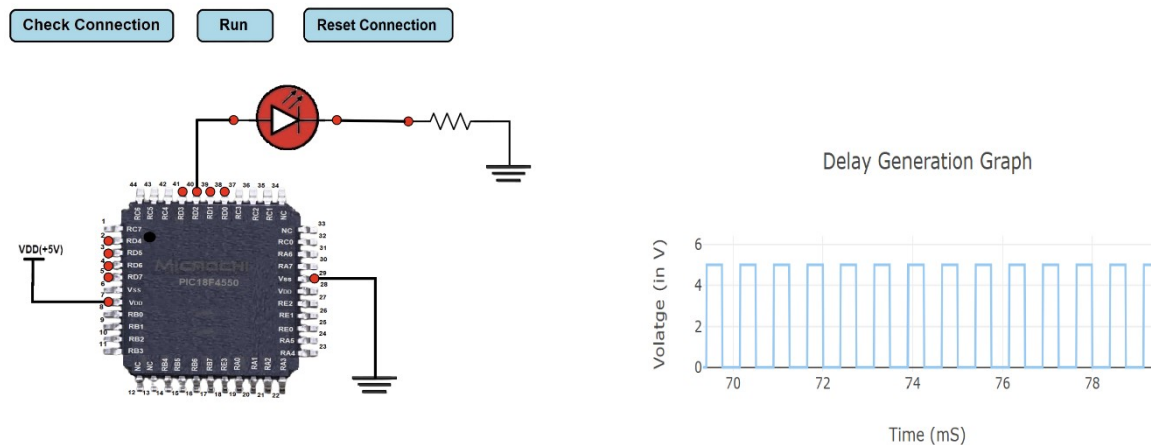
The generated waveform is a square wave with a total period of 400 ms. The output remains HIGH (LED ON) for 100 ms and LOW (LED OFF) for 300 ms in each cycle. Since the OFF delay is greater than the ON delay, the LOW pulse is wider than the HIGH pulse, resulting in the LED staying OFF for a longer duration than it stays ON.

CONFIGURATION	
Select Clock Frequency:	2 MHz
Enter Total Delay(ms):	400
Enter ON-OFF Delay(ms):	100 300 SUBMIT
Select Pin:	RD6
Select LED Configuration:	Current Sourcing

OUTPUT 3:**INPUT CONFIGURATION 4:**

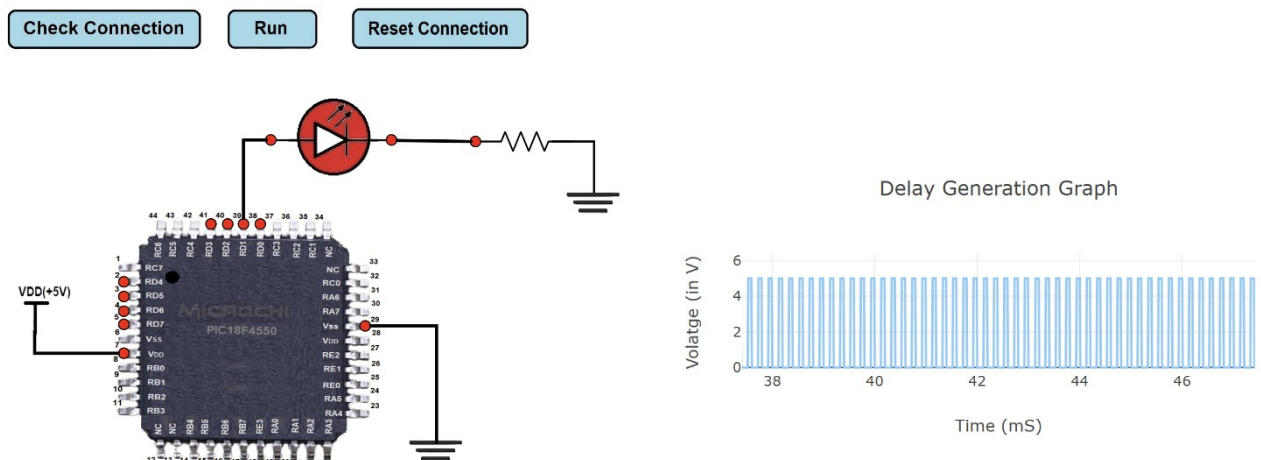
The generated waveform is a square wave with a total period of 750 ms. The output remains HIGH (LED ON) for 350 ms and LOW (LED OFF) for 400 ms in each cycle. Since the OFF delay is slightly greater than the ON delay, the LOW pulse is slightly wider than the HIGH pulse, resulting in the LED staying OFF slightly longer than it stays ON.

CONFIGURATION	
Select Clock Frequency:	20 MHz
Enter Total Delay(ms):	750
Enter ON-OFF Delay(ms):	350 400 SUBMIT
Select Pin:	RD2
Select LED Configuration:	Current Sourcing

OUTPUT 4:**INPUT CONFIGURATION 5:**

The generated waveform is a square wave with a total period of 200 ms. The output remains HIGH (LED ON) for 70 ms and LOW (LED OFF) for 130 ms in each cycle. Since the OFF delay is greater than the ON delay, the LOW pulse is wider than the HIGH pulse, resulting in the LED staying OFF longer than it stays ON.

CONFIGURATION	
Select Clock Frequency:	1 MHz
Enter Total Delay(ms):	200
Enter ON-OFF Delay(ms):	70 130
Select Pin:	RD1
Select LED Configuration:	Current Sourcing

OUTPUT 5:**RESULT:**

The LED blinking program was successfully executed on the PIC18F4550 microcontroller. The LED blinked according to the specified ON and OFF delays, and the delay generation graph matched the configured timing parameters.

EXP NO.1b	<u>VIRTUAL LABS</u> PROGRAMMING FOR I/O INTERFACING (LED AND SWITCH INTERFACING)
DATE: 03.07.26	

AIM:

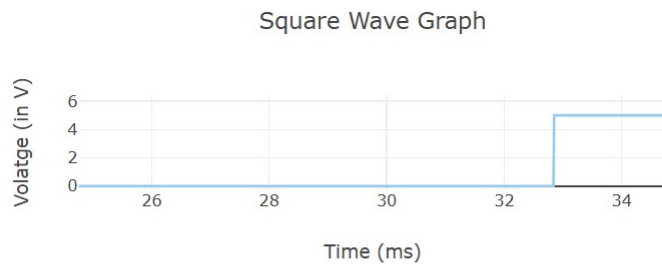
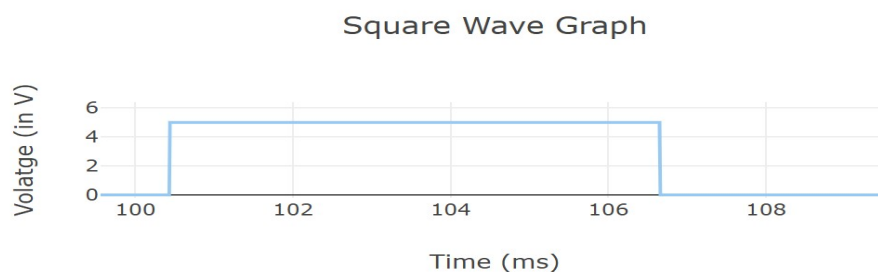
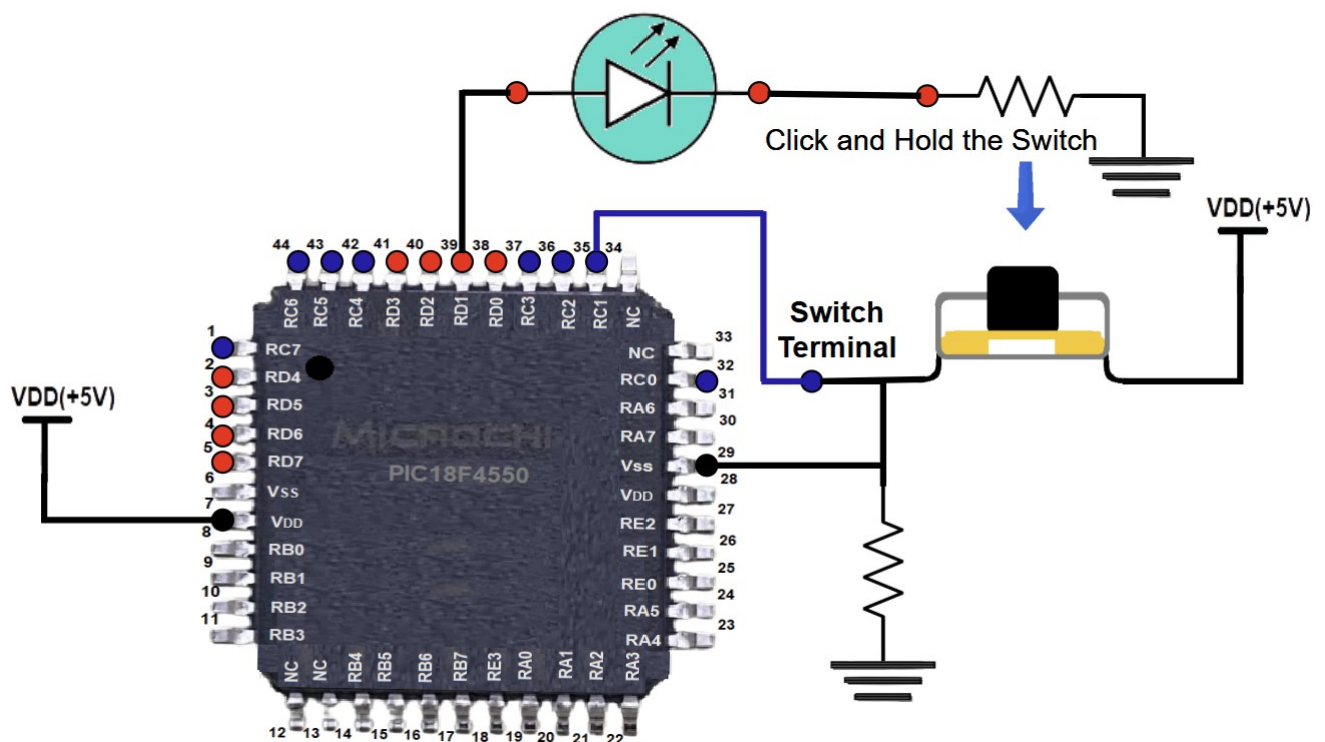
Write a program of LED blinking through push button using PIC18 series microcontroller

PROCEDURE:**Simulator :**

- step 1 : Select Input Pin For Switch.
- step 2 : Select Output Pin For LED.
- step 3 : Select Switch Configuration.
- step 4 : Select LED Configuration.
- step 5 : Click on "GENERATE CODE" button to generate code.
- step 6 : Click on "COMPILE" button to compile code.
- step 7 : Click on "EXECUTE" button to execute code and goto mimic.
- step 8 : If you want to download code click on "DOWNLOAD" button.
- step 9 : The MIMIC will open on left side of screen.
- step 10 : Connect the Output Pin to the Anode.
- step 11 : Click on the cathode terminal (red circle) and then click on the resistor terminal (red circle) to connect the cathode with the resistor or vice versa.
- step 12 : Click on the selected input pin terminal (blue circle) and then click on the switch terminal (blue circle) to connect the input pin with the switch terminal or vice versa.
- step 13 : Click on "Check Connection" button.
- step 14 : If connection is wrong, click on "Reset Connection" button and follow steps 10, 11, 12 and 13.
- step 15 : If connection is successful, click on Switch available on the screen with black color and hold the switch for a few seconds.
- step 16 : While holding the switch, Output LED is on. Observe the square wave graph to see how the output changes until you complete the observation.
- step 17 : Click on "REFRESH" button to re-executing the experiment.

SIMULATION:**INPUT CONFIGURATION:**

CONFIGURATION	
Select Input Pin For Switch :	RC1
Select Output Pin For LED :	RD2
Select Switch Configuration :	Press-to-High
Select LED Configuration :	Current Sourcing

OUTPUT:**Check Connection****Reset Connection****RESULT:**

The LED and switch interfacing program for the PIC18 series microcontroller was successfully implemented. The LED turned ON when the push button was pressed and OFF when the push button was released.

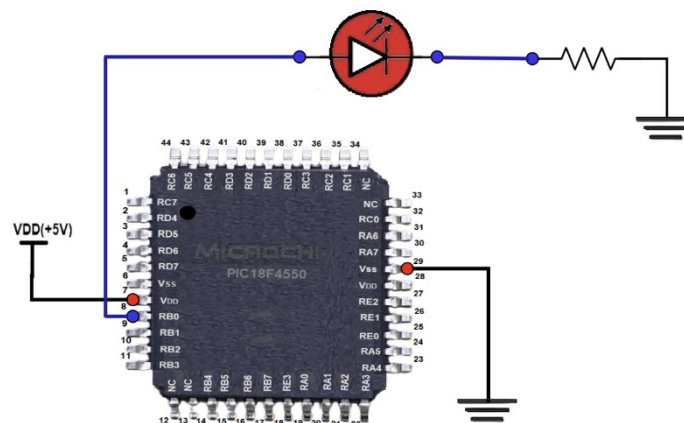
EXP NO.1c	<u>VIRTUAL LABS</u> PROGRAMMING WITH ON-CHIP TIMERS/COUNTERS
DATE: 10.07.26	

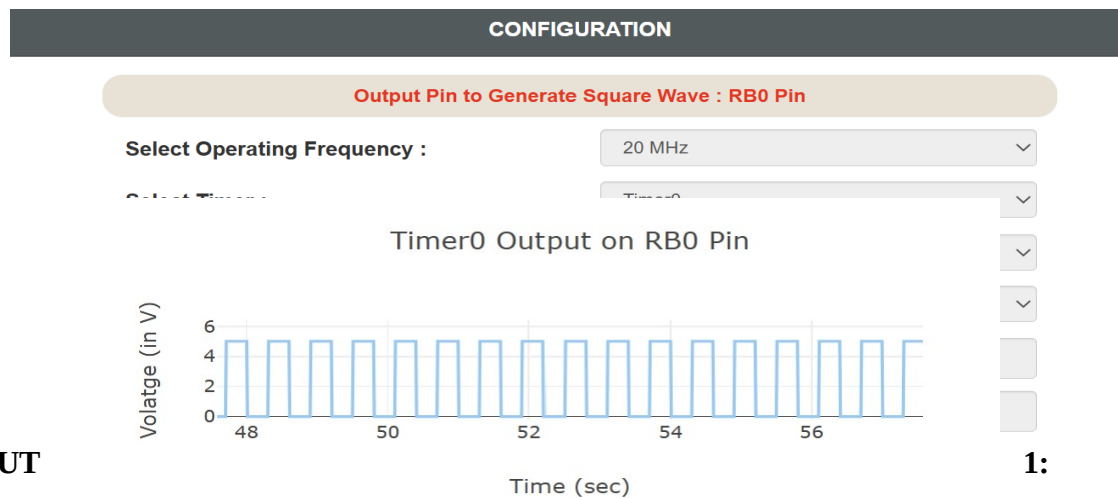
AIM:

Programming with on-chip Timers/Counters.

PROCEDURE:**Simulator:**

- step 1 : Select Operating Frequency.
- step 2 : Select Timer.
- step 3 : Select Prescaler.
- step 4 : Select Enable Interrupt.
- step 5 : Enter ON Delay (in mS).
- step 6 : Enter OFF Delay (in mS).
- step 7 : Click on "GENERATE CODE" button to generate code.
- step 8 : Click on "COMPILE" button to compile code.
- step 9 : Click on "EXECUTE" button to execute code and goto mimic.
- step 10 : If you want to download code click on "DOWNLOAD" button.
- step 11: The MIMIC will open on left side of screen.
- step 12 : Click on selected output pin terminal(blue circle) and click on anode terminal(blue circle) for connection of the selected output pin with anode or vice versa.
- step 13 : Click on cathode terminal(blue circle) and click on resistor terminal(blue circle) for connection of cathode with resistor or vice versa.
- step 14 : Click on "Check Connection" button.
- step 15 : If connection is wrong, click on "Reset Connection" button and follow step 10, 11 and 12.
- step 16 : If connection is successful, click on Run Button.
- step 17 : Output LED is on. Observe the square wave graph.
- step 18 : Click on "REFRESH" button to re-executing the experiment.

CIRCUIT DIAGRAM:

SIMULATION:**INPUT 1:****OUTPUT****INPUT 2:**

CONFIGURATION

Output Pin to Generate Square Wave : RB0 Pin

Select Operating Frequency : 20 MHz

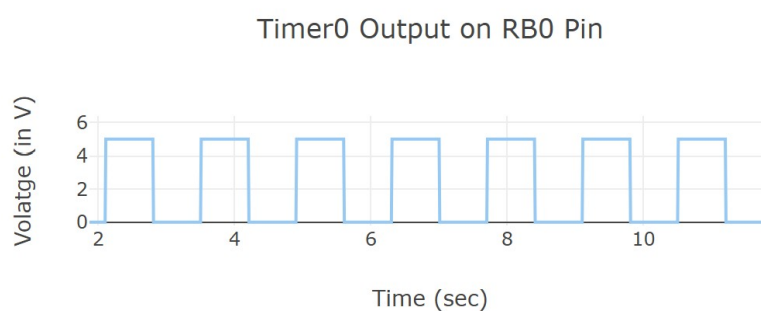
Select Timer : Timer0

Select Prescaler : 1:256

Select Enable Interrupt : YES

Enter ON Delay (in ms) : 700

Enter OFF Delay (in ms) : 700

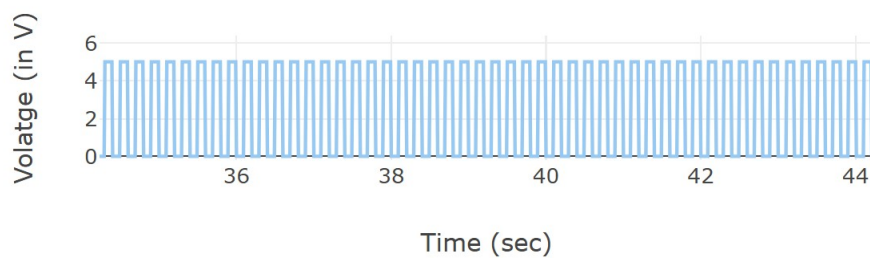
OUTPUT 2:

INPUT 3:

CONFIGURATION	
Output Pin to Generate Square Wave : RB0 Pin	
Select Operating Frequency :	20 MHz
Select Timer :	Timer0
Select Prescaler :	1:32
Select Enable Interrupt :	YES
Enter ON Delay (in ms) :	100
Enter OFF Delay (in ms) :	100

OUTPUT 3:

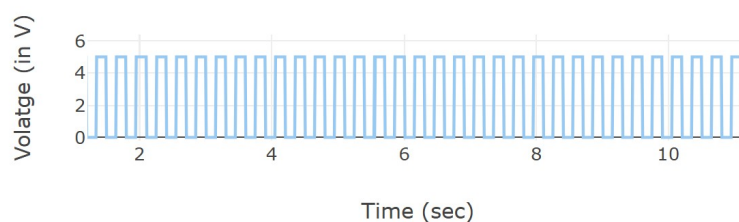
Timer0 Output on RB0 Pin

**INPUT 4:**

CONFIGURATION	
Output Pin to Generate Square Wave : RB0 Pin	
Select Operating Frequency :	20 MHz
Select Timer :	Timer0
Select Prescaler :	1:64
Select Enable Interrupt :	YES
Enter ON Delay (in ms) :	150
Enter OFF Delay (in ms) :	150

OUPUT 4:

Timer0 Output on RB0 Pin

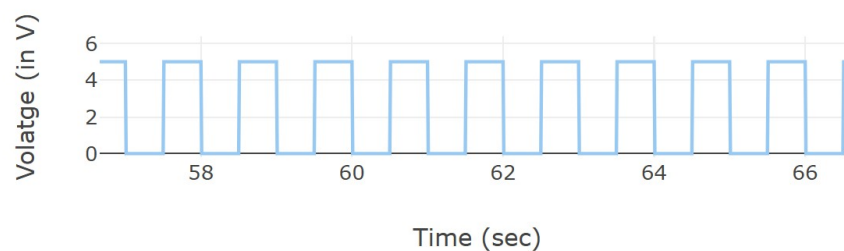


INPUT 5:**CONFIGURATION****Output Pin to Generate Square Wave : RB0 Pin**

Select Operating Frequency :	48 MHz
Select Timer :	--- Select Timer ---
Select Prescaler :	1:16
Select Enable Interrupt :	YES
Enter ON Delay (in ms) :	500
Enter OFF Delay (in ms) :	500

OUTPUT 5:

Timer0 Output on RB0 Pin

**RESULT:**

The Timer and Counter module was successfully implemented and verified.

EXP NO.2	STUDY OF KEIL SOFTWARE
DATE: 17.07.26	

AIM:

To write, compile, simulate, and execute a C program using Keil μ Vision for the AT89C51 microcontroller to set all the pins of Port 1 to logic HIGH.

KEIL SOFTWARE:

Keil μ Vision is an Integrated Development Environment (IDE) developed for embedded system programming. It provides all the necessary tools required to develop applications for microcontrollers, including a code editor, compiler, assembler, linker, debugger, and simulator. Keil supports several microcontroller families such as 8051, ARM Cortex-M, C166, and 251. For 8051 development, Keil uses the C51 Compiler, which converts C programs into machine code executable by the AT89C51 microcontroller. The IDE also provides a built-in simulator that enables users to test programs without requiring physical hardware.

The debugger allows programmers to execute the program step-by-step, inspect register values, monitor memory locations, and observe port outputs. The generated executable can also be converted into a HEX file, which is programmed into the microcontroller using a suitable programmer.

COMPONENTS OF KEIL μ VISION:**1. Code Editor**

- Used to write C and Assembly programs.
- Supports syntax highlighting.
- Provides automatic indentation.

2. Compiler

- Converts C source code into object code.
- Reports syntax and logical errors.

3. Assembler

- Converts Assembly language into machine language.

4. Linker

- Combines object files and libraries.
- Generates the executable program.

5. Debugger

- Executes the program line by line.
- Displays register values.
- Supports breakpoints.
- Allows variable monitoring.

6. Simulator

- Simulates the complete 8051 microcontroller.
- Displays Port 0, Port 1, Port 2, and Port 3.
- No hardware is required.

FEATURES OF KEIL μ VISION:

1. Supports C and Assembly Programming:

Keil supports programming in both Embedded C and Assembly Language, allowing developers to choose the most suitable language based on application requirements. Embedded C simplifies application development by providing high-level programming constructs, while Assembly Language offers direct control over the hardware and efficient memory utilization. Both languages can be used together within the same project, making Keil highly flexible for embedded system design.

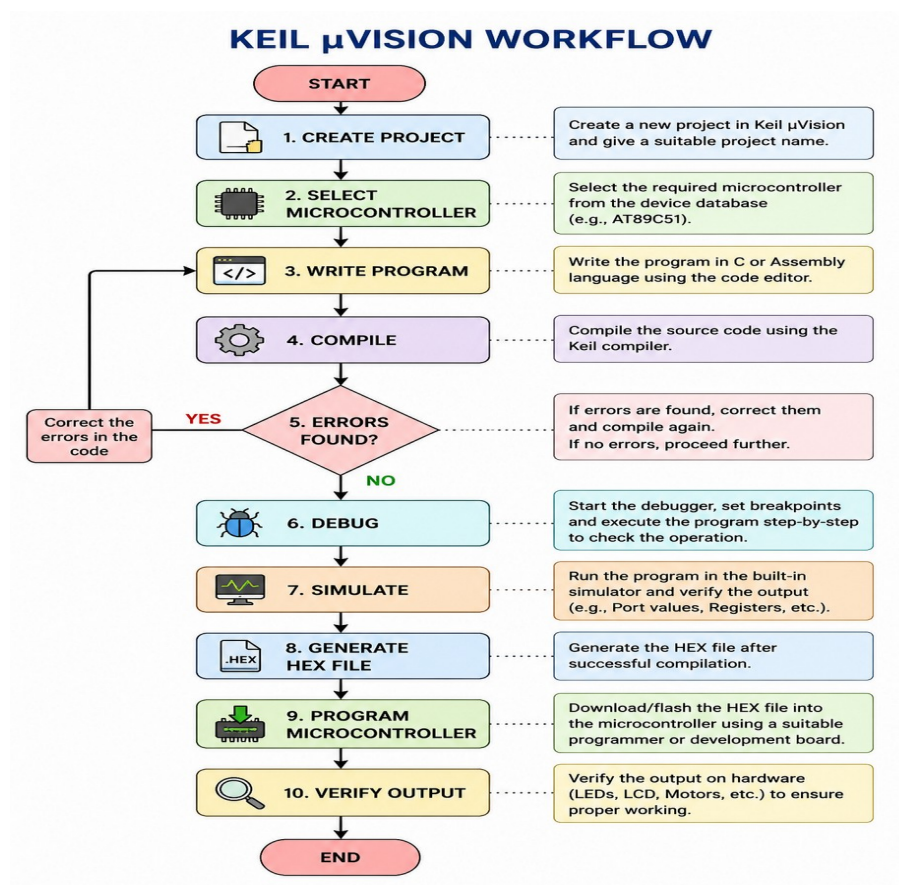
2. Integrated Compiler and Debugger

The Keil IDE includes a powerful compiler that translates C source code into machine code suitable for the selected microcontroller. It also features an integrated debugger that helps developers identify and correct programming errors. The debugger allows users to execute programs line by line, inspect register contents, monitor variables, examine memory locations, and set breakpoints for efficient troubleshooting. This integrated environment significantly reduces debugging time.

3. Fast Compilation

Keil provides a highly optimized compiler that performs rapid compilation of source code into executable machine code. Even large embedded projects can be compiled quickly, enabling developers to identify and correct programming errors efficiently. Faster compilation shortens the software development cycle and increases overall productivity during testing and debugging.

KEIL WORKFLOW:

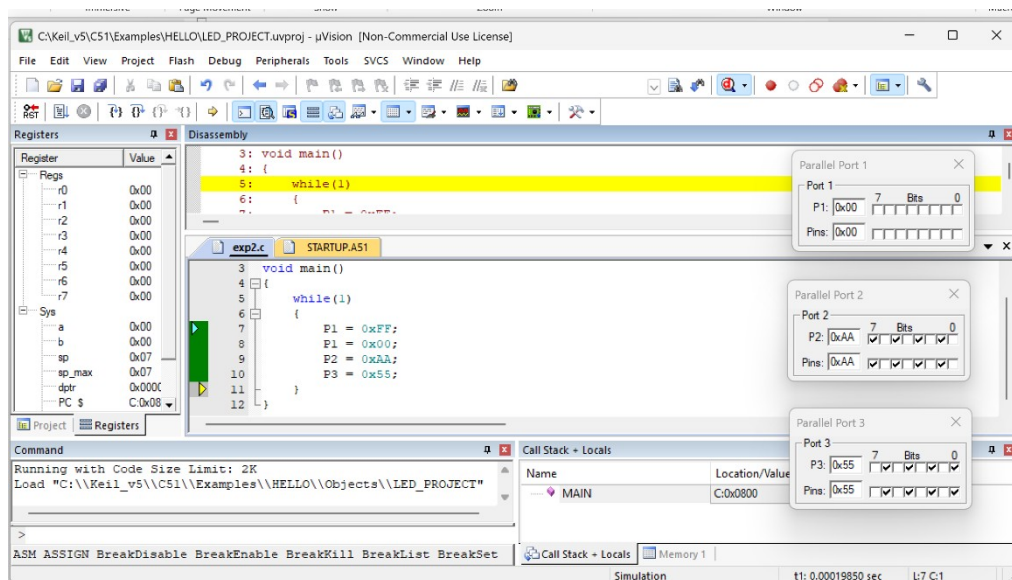


PROGRAM:

```
#include <reg51.h>

void main()
{
    while(1)
    {
        P1 = 0xFF;
        P1 = 0x00;

        P2 = 0xAA;
        P3 = 0x55;
    }
}
```

OUTPUT:**Result**

The Keil uVision IDE was successfully studied. The project was created, compiled, debugged, simulated, and the output was verified using the Parallel Port window.

EXP NO.3	ARITHMETIC AND LOGICAL OPERATIONS USING 8051 ASSEMBLY LANGUAGE IN KEIL μVISION
DATE: 24.07.26	

AIM:

To write, compile, simulate, and verify arithmetic and logical operations using 8051 Assembly Language (.A51) in Keil μ Vision IDE.

DESCRIPTION:

Arithmetic and logical operations are the fundamental instructions of every microcontroller. The 8051 microcontroller performs these operations using its Arithmetic Logic Unit (ALU). The ALU processes data stored in the Accumulator (Register A) and other registers to perform mathematical calculations and bitwise manipulations.

In the 8051 Assembly Language, arithmetic instructions such as ADD, SUBB, MUL, and DIV are used for performing mathematical operations. Similarly, logical instructions such as ANL (AND), ORL (OR), XRL (Exclusive OR), and CPL (Complement) are used for manipulating binary data. These instructions are widely used in embedded systems for decision making, data processing, masking bits, setting or clearing bits, and controlling hardware peripherals. The program is developed using Keil μ Vision IDE, assembled into machine code, and executed using the built-in simulator. The results of each instruction are verified by observing the contents of the Accumulator, Carry Flag, Overflow Flag, and General Purpose Registers (R0–R7).

PROGRAM:**ARITHMETIC OPERATIONS****1 (i): Addition without Carry**

Objective: To add two hexadecimal numbers whose sum does not generate a carry.

Input:

- 03H = 3
- 05H = 5

Calculation:

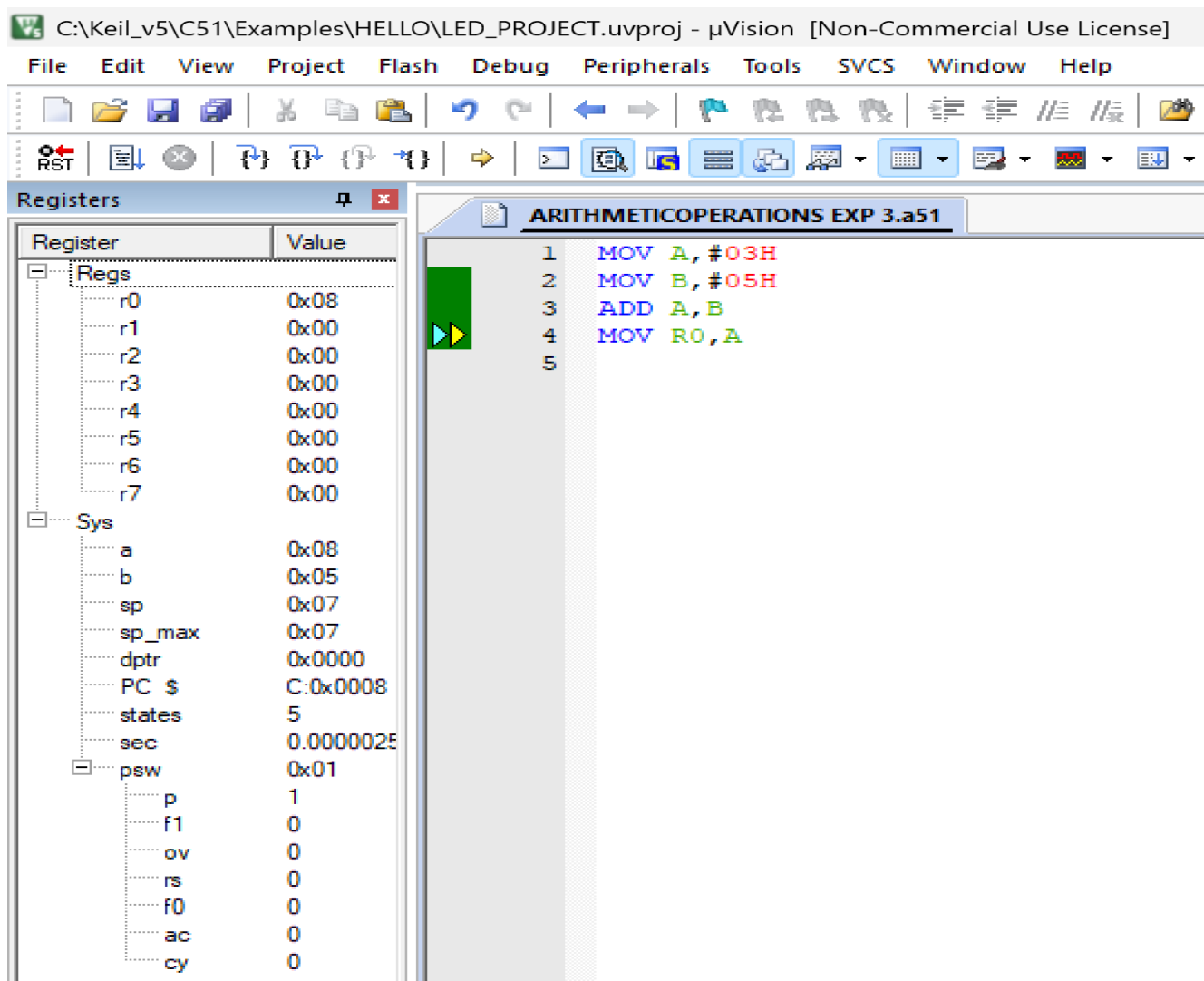
- $3 + 5 = 8$

Result:

- R8 = 08H
- Carry = 0

Instruction:

```
MOV A,#03H
MOV B,#05H
ADD A,B
MOV R0,A
```

OUTPUT:**1 (ii): Addition with Carry**

Objective: To add two hexadecimal numbers whose sum generates a carry.

Input:

- F0H = 240
- 20H = 32

Result:

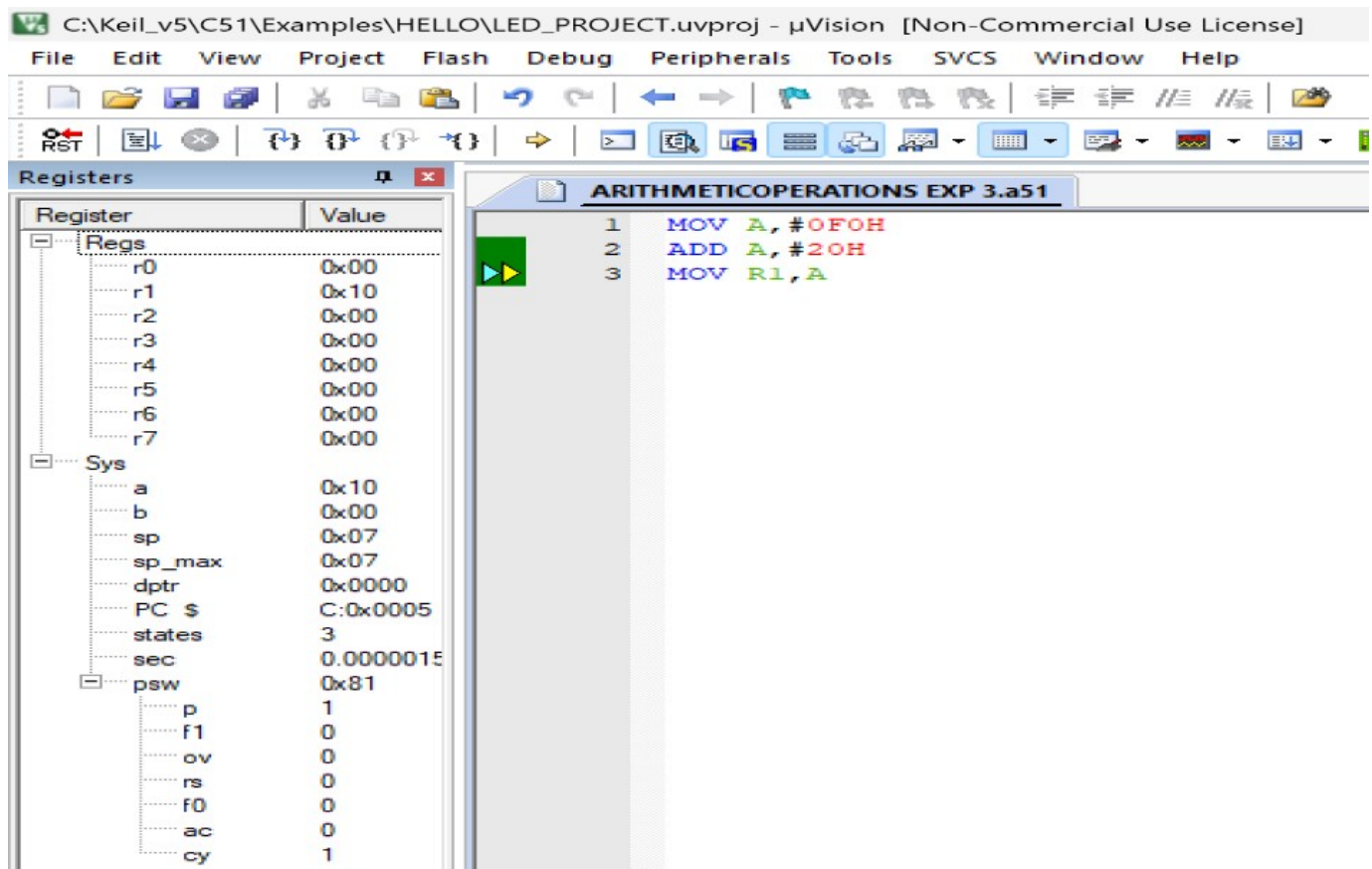
- Register R1 = 10H (16)
- Carry = 1

Instruction:

```

CLR C
MOV A, #0F0H
ADD A, #20H
MOV R1, A

```

OUTPUT:**2 (i) Subtraction without Borrow**

Objective: To subtract two hexadecimal numbers without borrow.

Input:

- 08H = 8
- 03H = 3

Result:

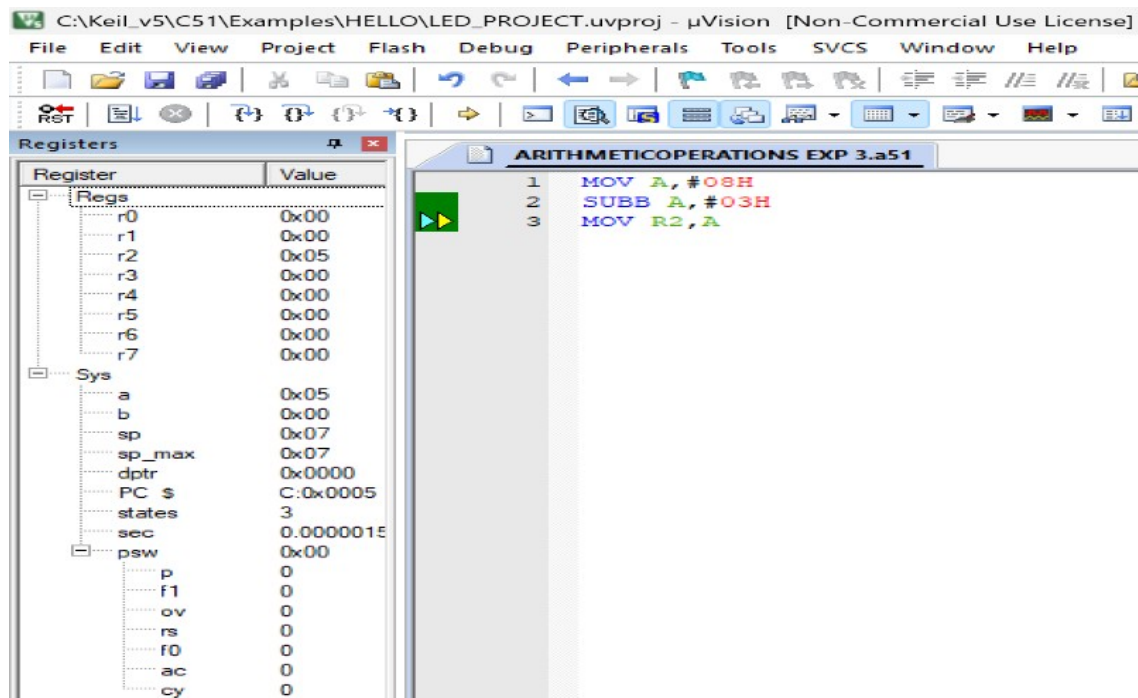
- 05H = 5
- Borrow (Carry) = 0

Instruction:

```

CLR C
MOV A, #08H
SUBB A, #03H
MOV R2, A

```

Output:**2 (ii): Subtraction with Borrow**

Objective: To subtract a larger number from a smaller number.

Input:

- 03H = 3 , 05H = 5

Result:

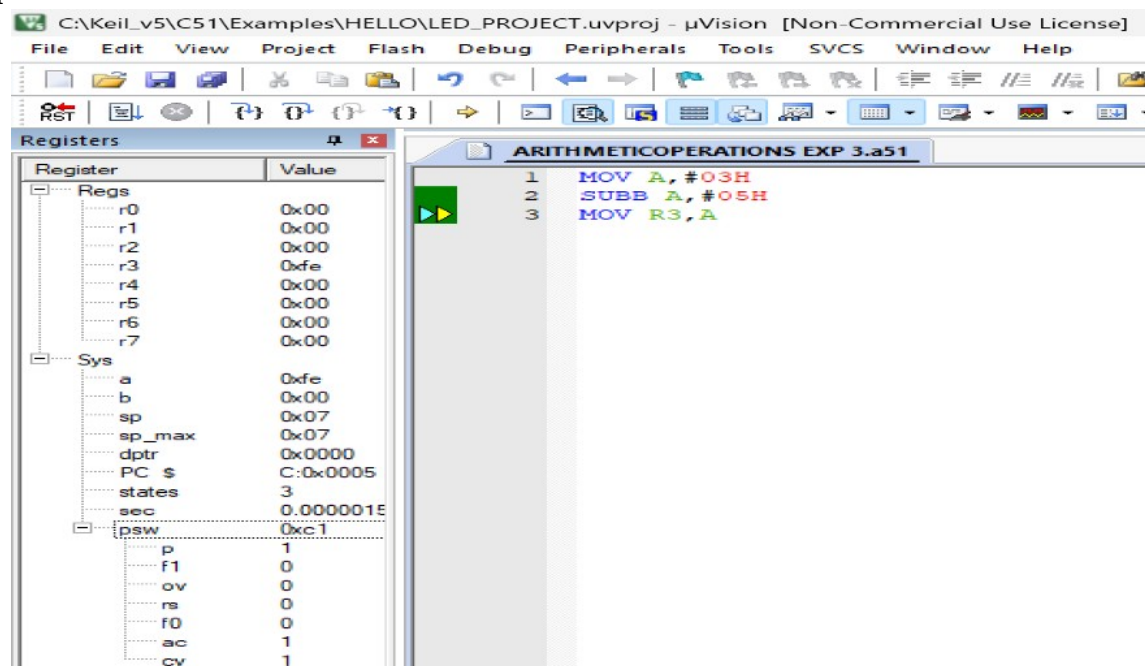
- FEH (2's Complement of -2) , Borrow (Carry) = 1

Instruction:

```

CLR C
MOV A, #03H
SUBB A, #05H
MOV R3, A
  
```

Output:



3 (i): Multiplication without Overflow**Objective:** To multiply two hexadecimal numbers without overflow.**Input:**

- 08H = 8 , 04H = 4

Result:

- 20H = 32 , B Register = 00H

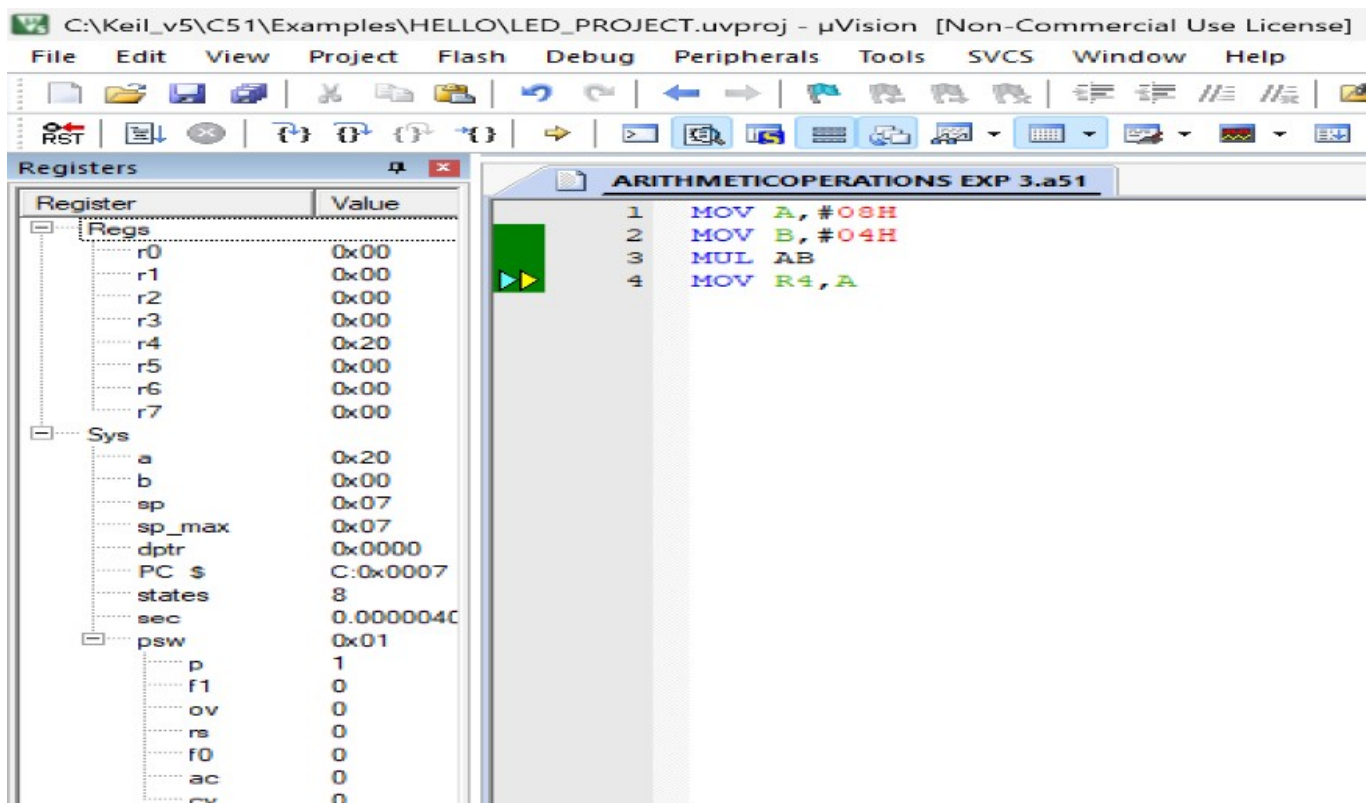
Instruction:

MOV A,#08H

MOV B,#04H

MUL AB

MOV R4,A

Output:**3 (ii): Multiplication with Overflow****Objective:** To multiply two hexadecimal numbers producing a 16-bit result.**Input:**

- 20H = 32 , 10H = 16

Result:

- 0200H , A = 00H (Lower Byte) , B = 02H (Higher Byte)

Instruction:

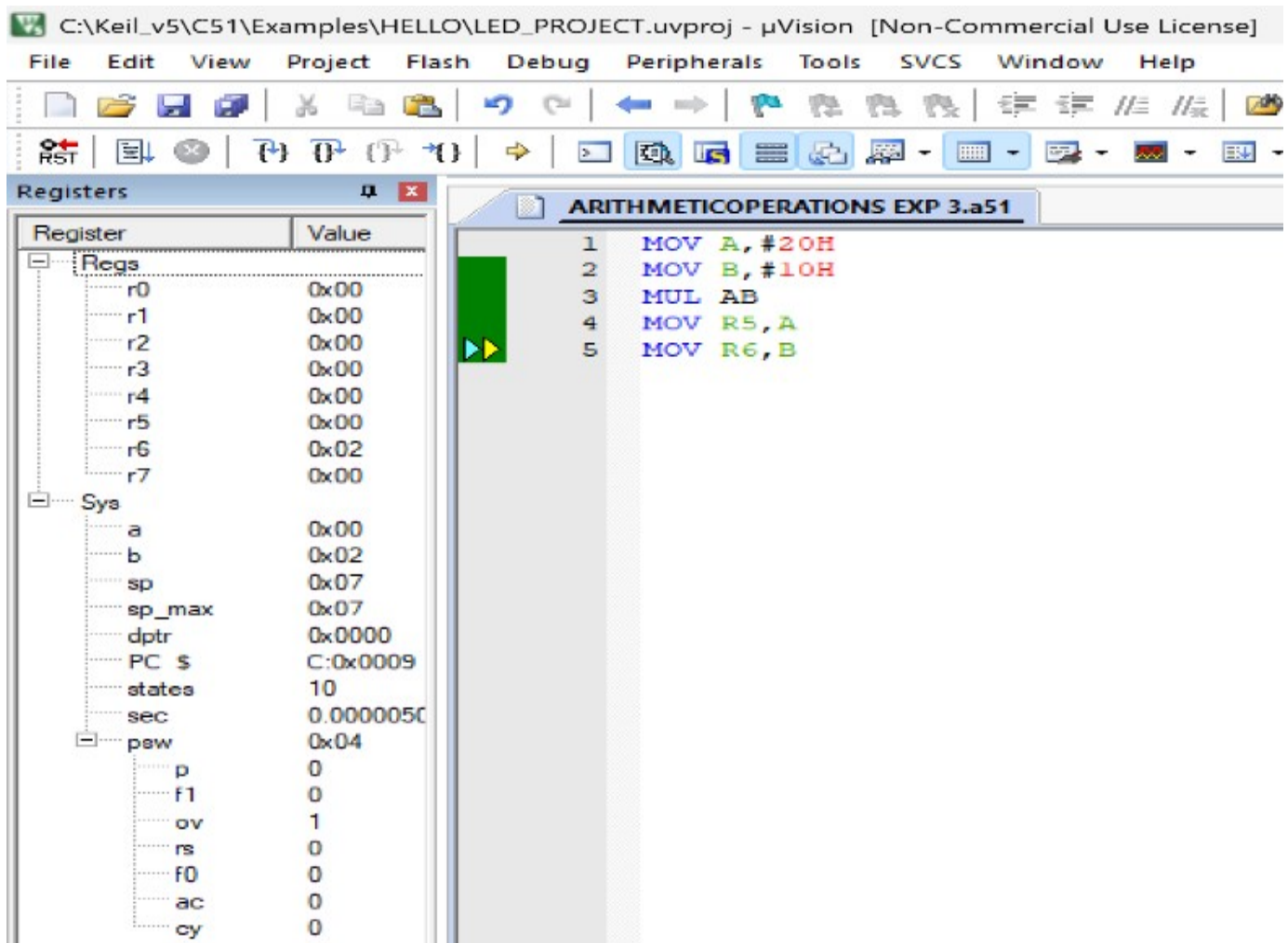
MOV A,#20H

MOV B,#10H

MUL AB

MOV R5,A

MOV R6,B

Output:**4 (i): Division without Remainder**

Objective: To divide two hexadecimal numbers without remainder.

Input:

- 20H = 32
- 04H = 4

Result:

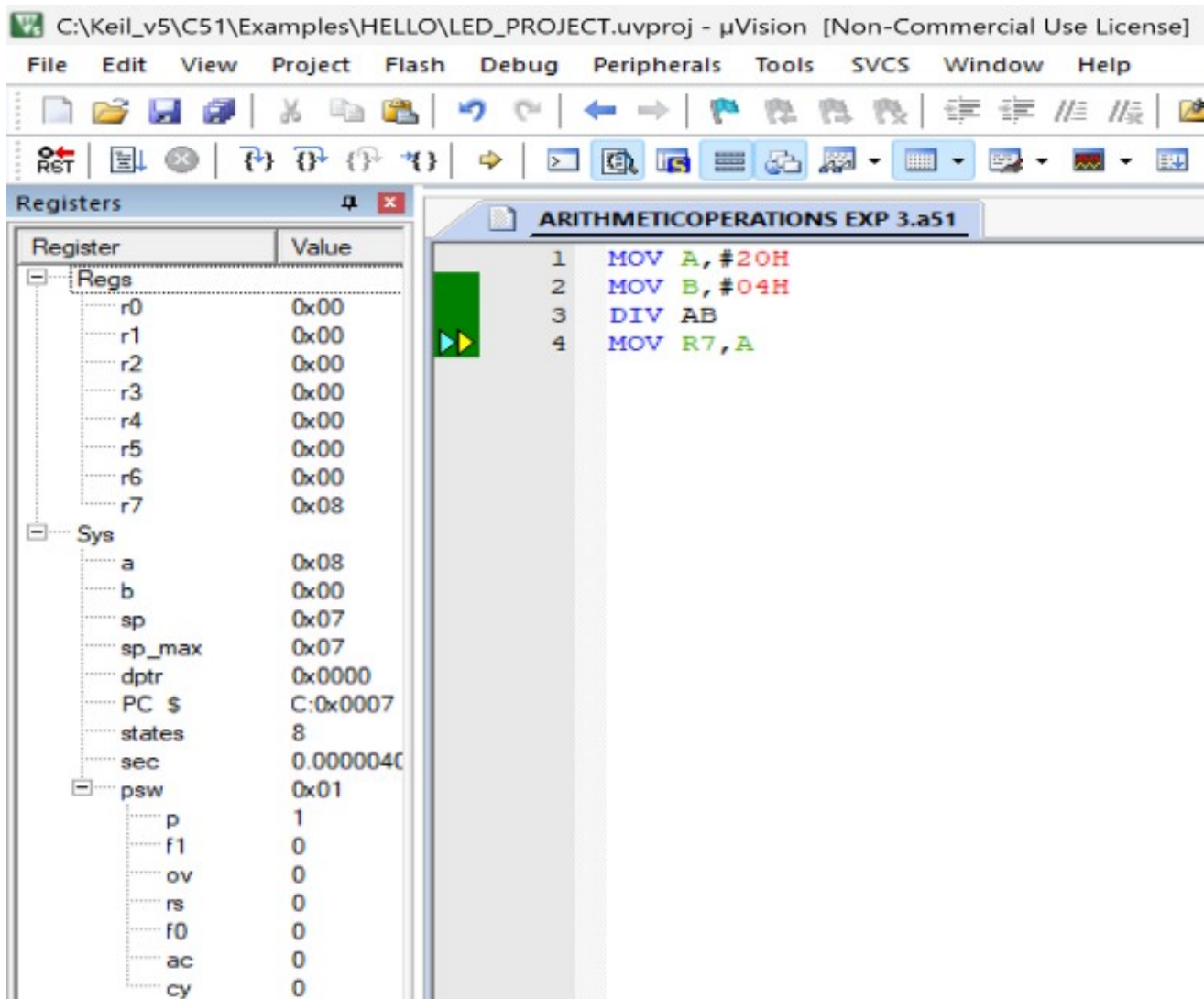
- Quotient = 08H
- Remainder = 00H

Instruction:

```

MOV A, #20H
MOV B, #04H
DIV AB
MOV R7, A

```

Output:**4 (ii): Division with Remainder**

Objective: To divide two hexadecimal numbers with remainder.

Input:

- 25H = 37
- 06H = 6

Result:

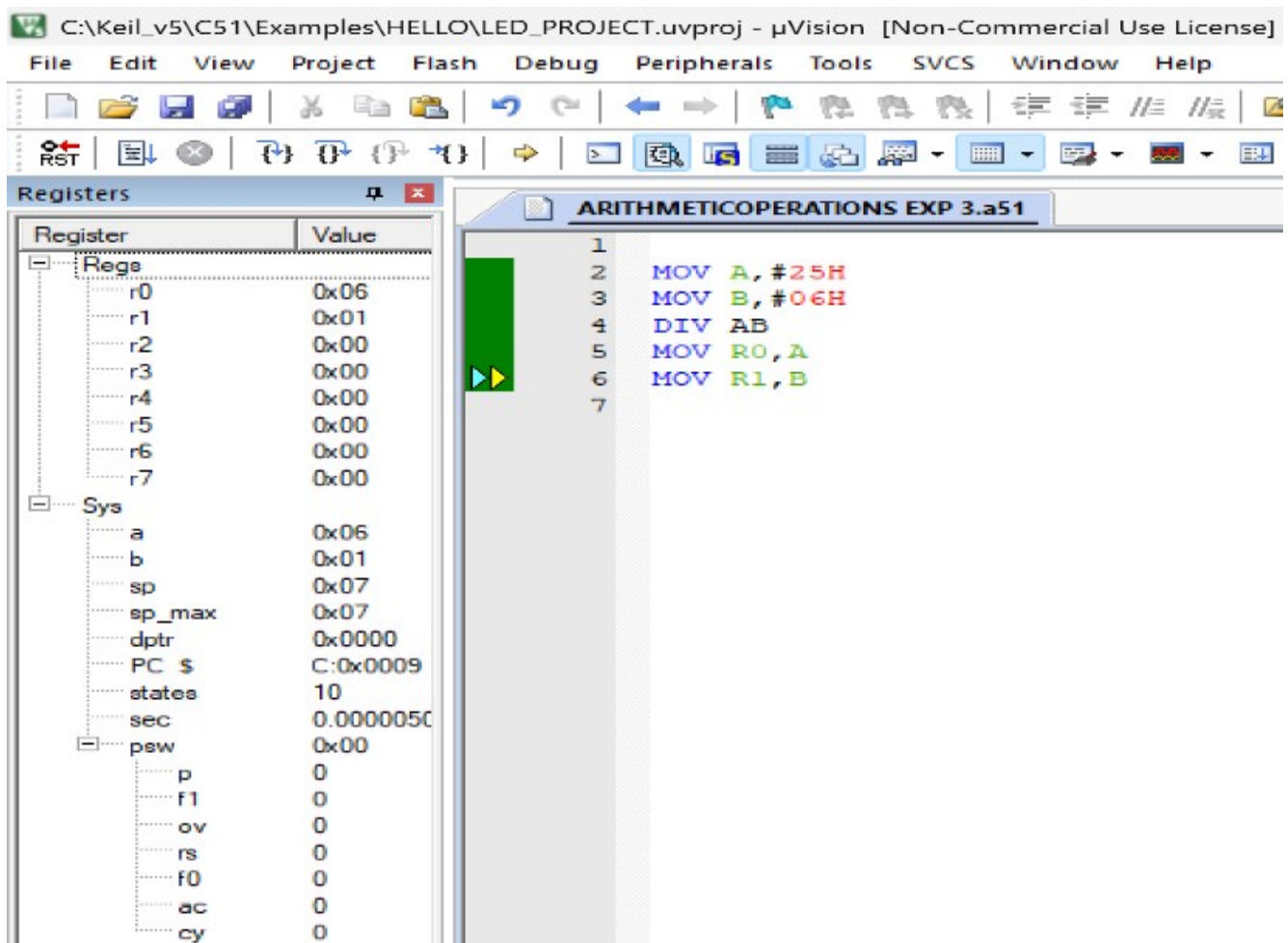
- Quotient = 06H
- Remainder = 01H

Instructuion:

```

MOV A, #25H
MOV B, #06H
DIV AB
MOV R0, A
MOV R1, B

```

Output:**LOGICAL OPERATIONS****5 (i): Logical AND**

Objective: To perform bitwise AND operation.

Input:

- 0FH = 15
- 05H = 5

Result:

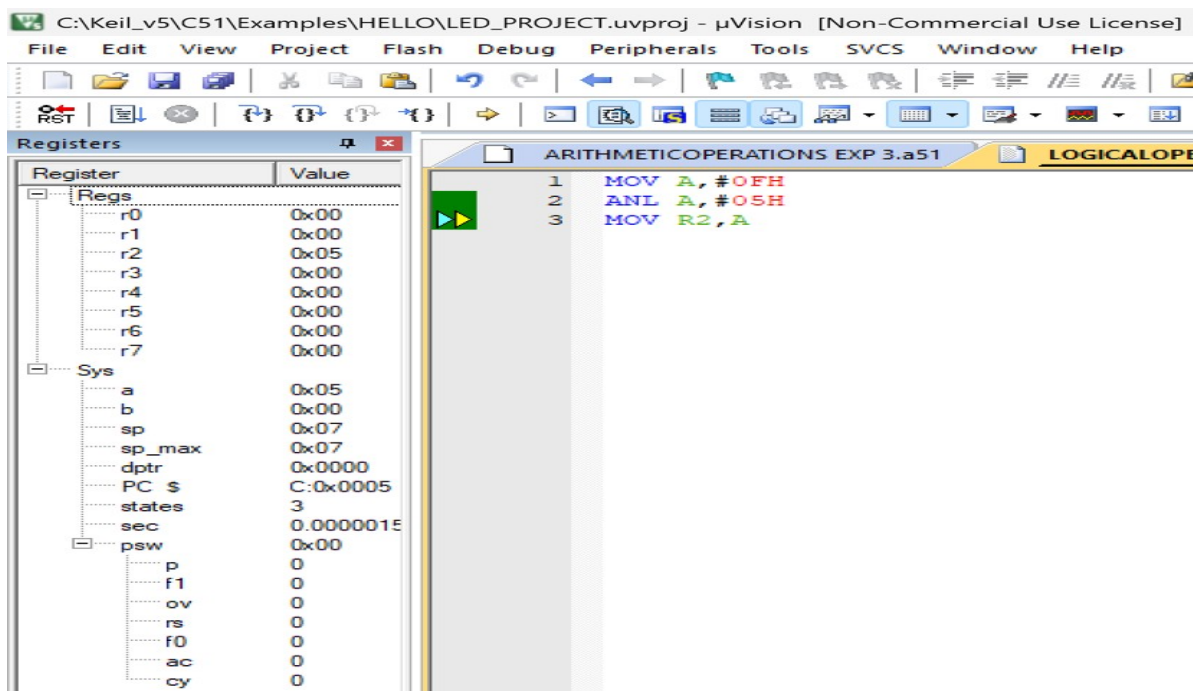
- 05H = 5

Instruction:

```

MOV A, #0FH
ANL A, #05H
MOV R2, A

```

Output:**5 (ii): Logical OR****Objective:** To perform bitwise OR operation.**Input:**

- 09H = 9
- 06H = 6

Result:

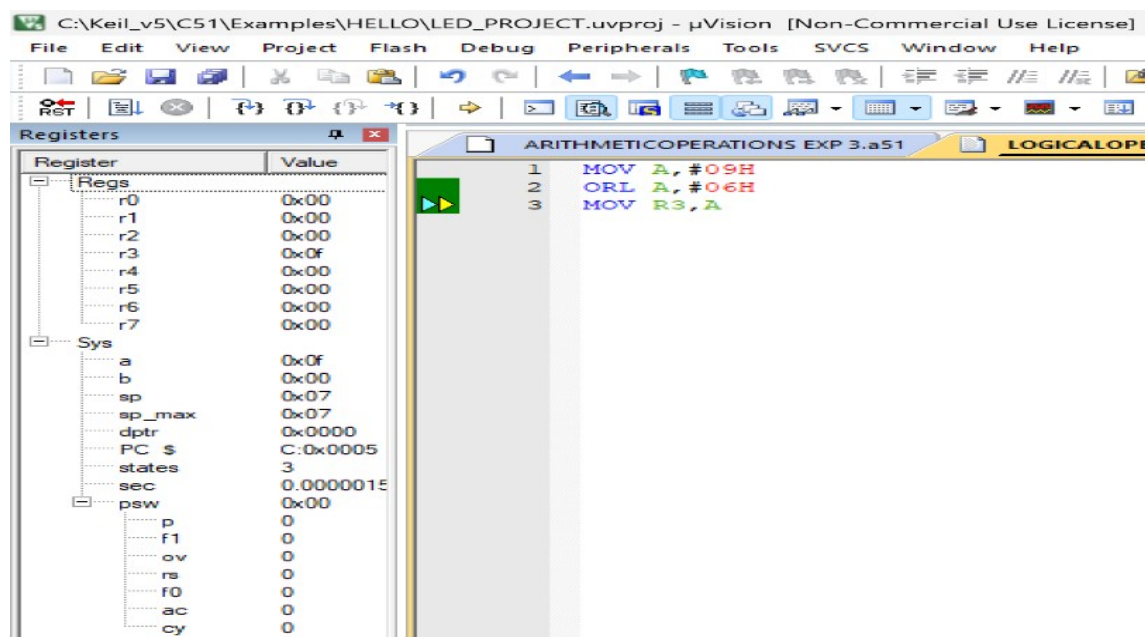
- 0FH = 15

Instruction:

```

MOV A,#09H
ORL A,#06H
MOV R3,A

```

Output:

5 (iii): Logical XOR**Objective:** To perform bitwise Exclusive-OR (XOR) operation.**Input:**

- 0FH = 15 , 05H = 5

Result:

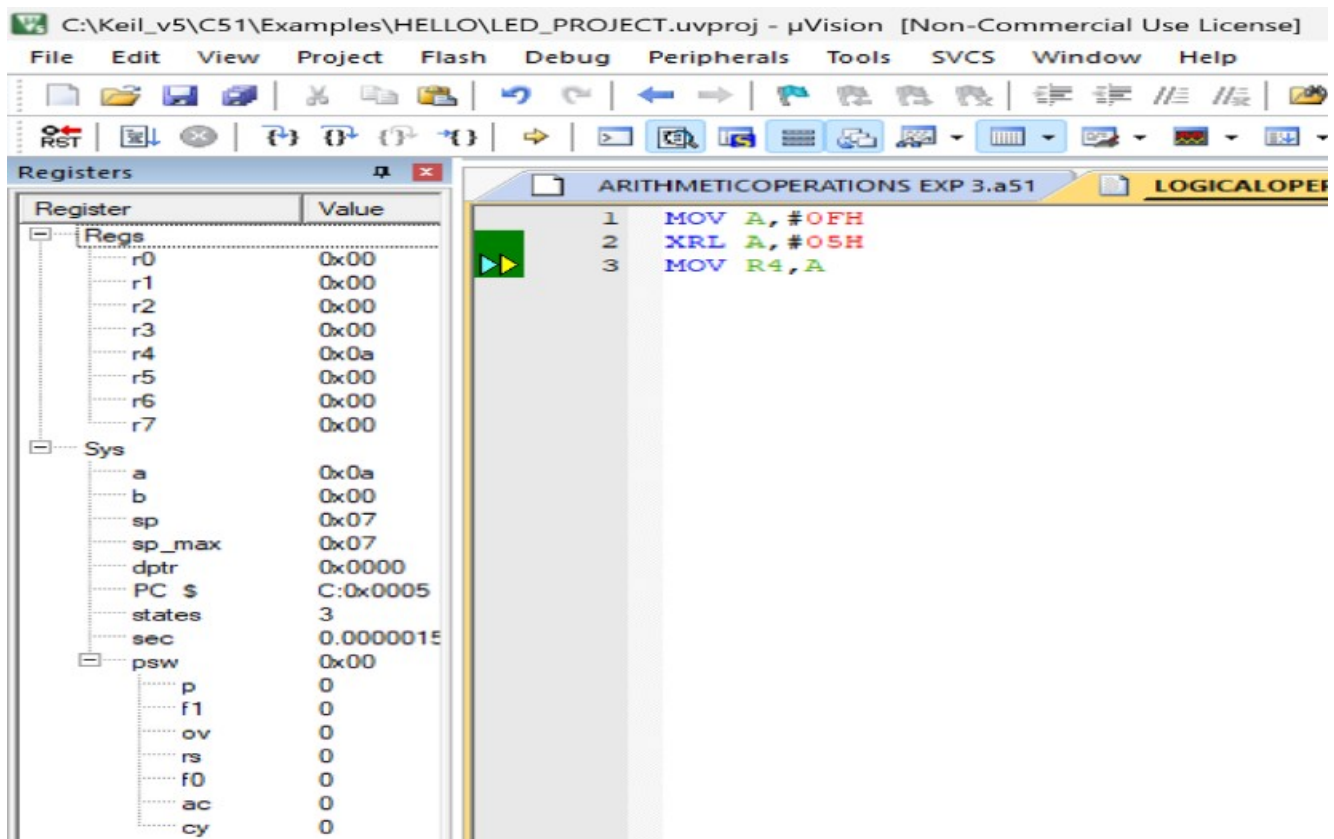
- 0AH = 10

Instruction:

MOV A,#0FH

XRL A,#05H

MOV R4,A

Output:**5 (iv): Complement****Objective:** To perform bitwise complement operation.**Input:**

- 55H = 85

Result:

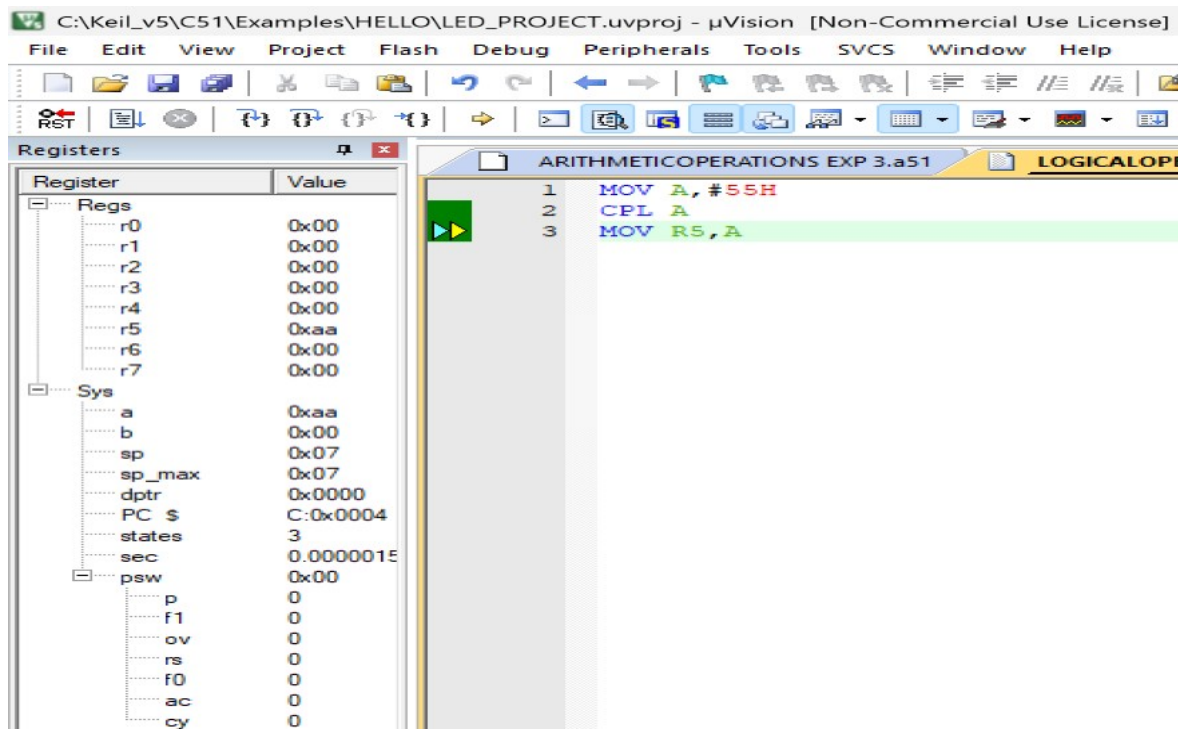
- AAH = 170

Instruction:

MOV A,#55H

CPL A

MOV R5,A

Output:**5 (v): Rotate Left**

Objective: To rotate all bits of the accumulator one position to the left.

Input:

- 81H = 129

Result:

- 03H = 3

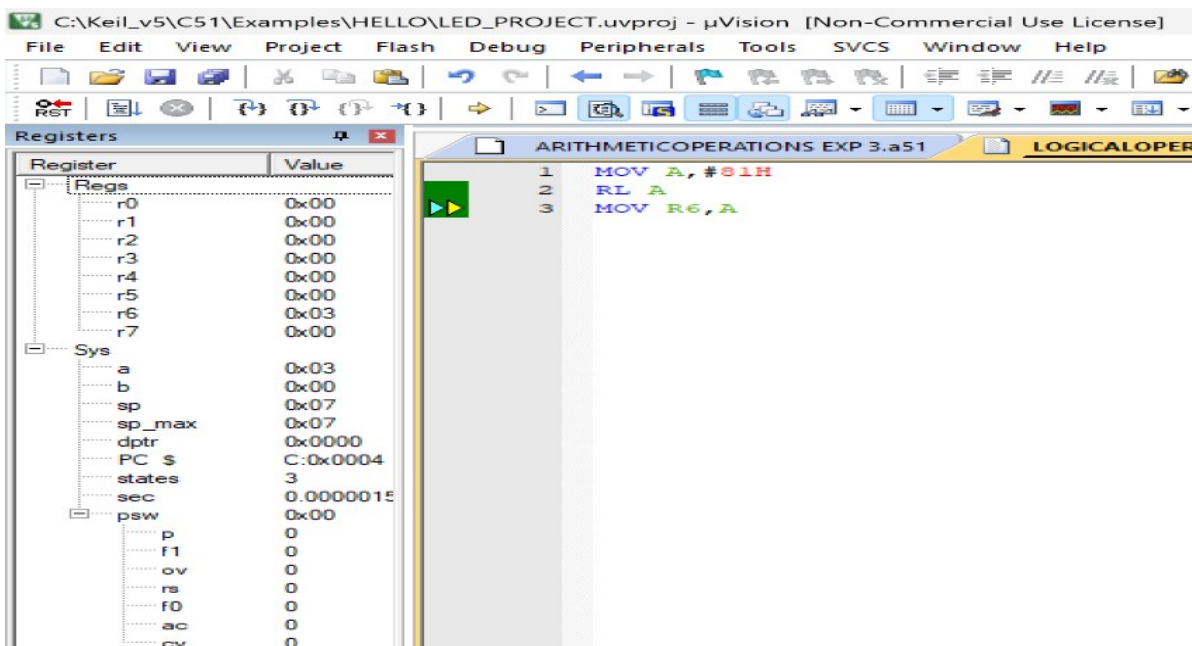
Instruction:

MOV A,#81H

RL A

MOV R6,A

Output:



5 (vi): Rotate Right**Objective:** To rotate all bits of the accumulator one position to the right.**Input:**

- 81H = 129

Result:

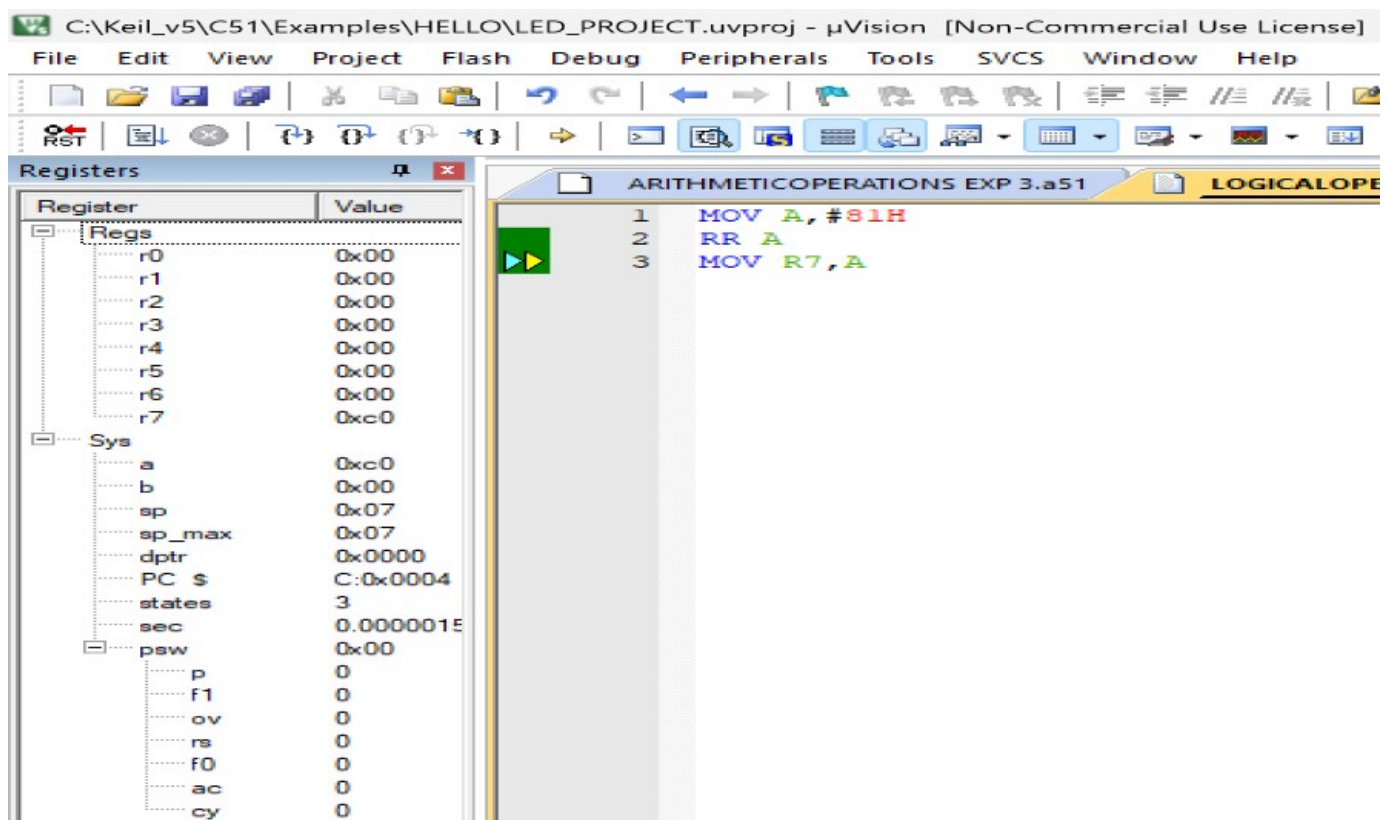
- C0H = 192

Instruction:

MOV A,#81H

RR A

MOV R7,A

Output:**5 (vii): Clear Accumulator****Objective:** To clear all bits of the Accumulator.**Input:**

- Accumulator (A) contains any value.

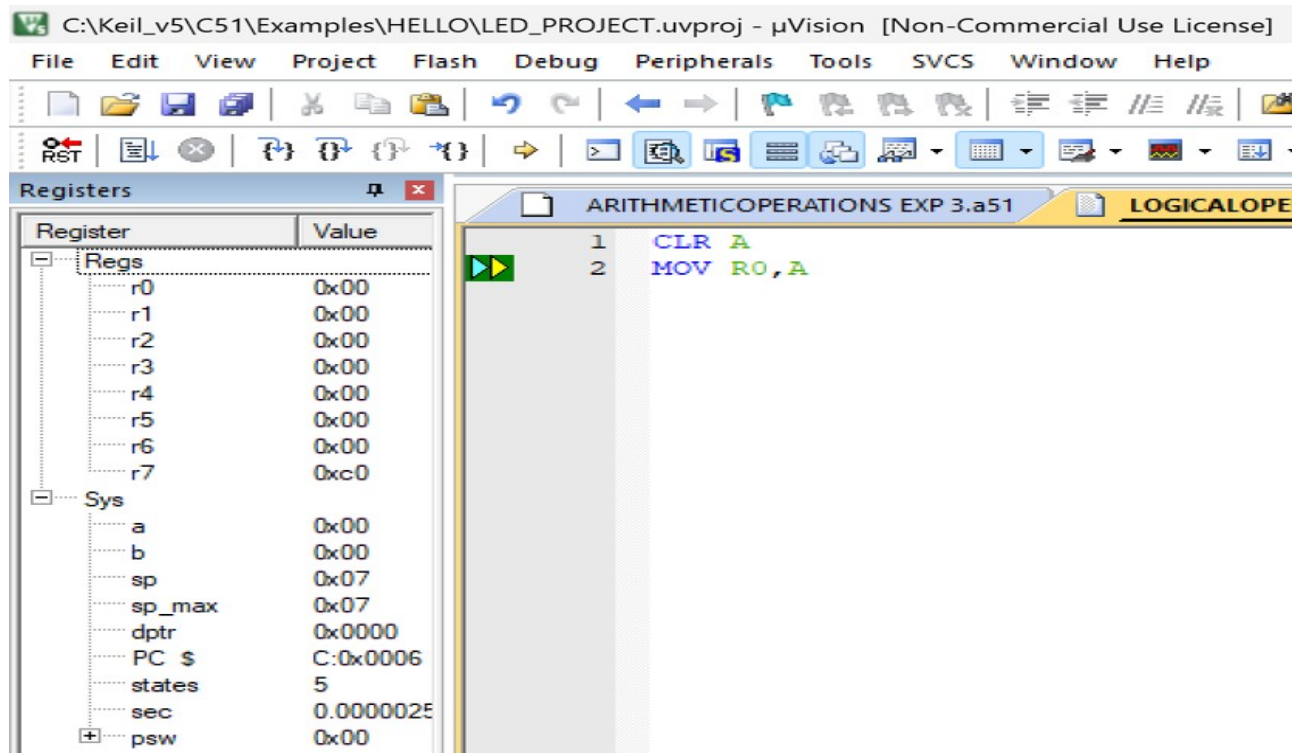
Result:

- 00H = 0

Instruction:

CLR A

MOV R0,A

Output:**5 (viii): Set Carry Flag**

Objective: To set the Carry flag to logic 1.

Input:

- Carry Flag = 0

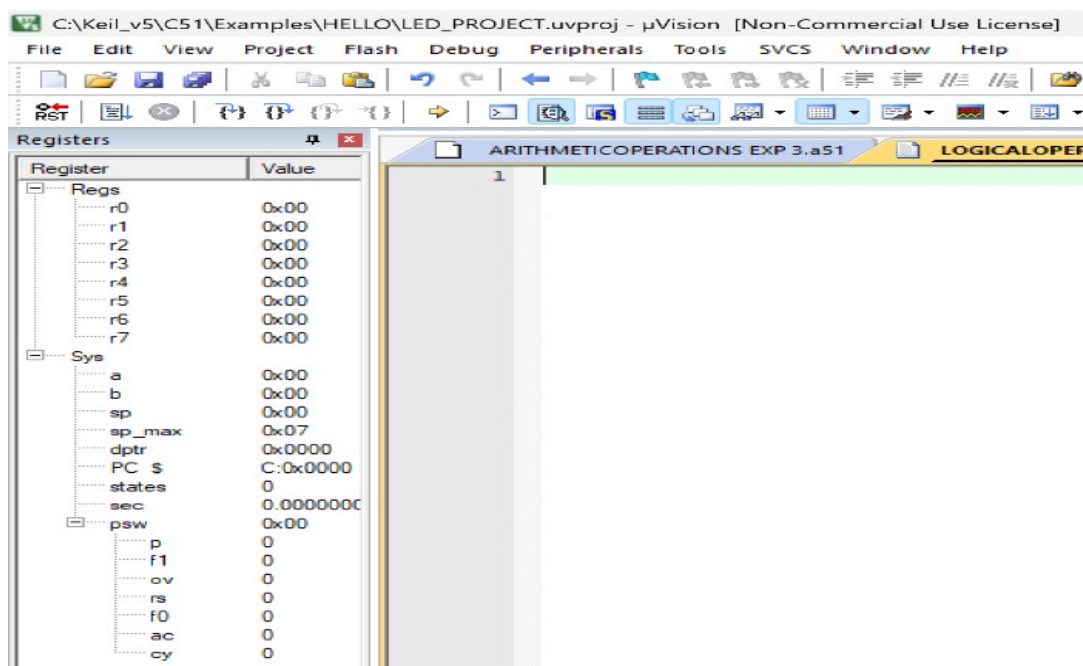
Result:

- Carry Flag = 1

Instruction:

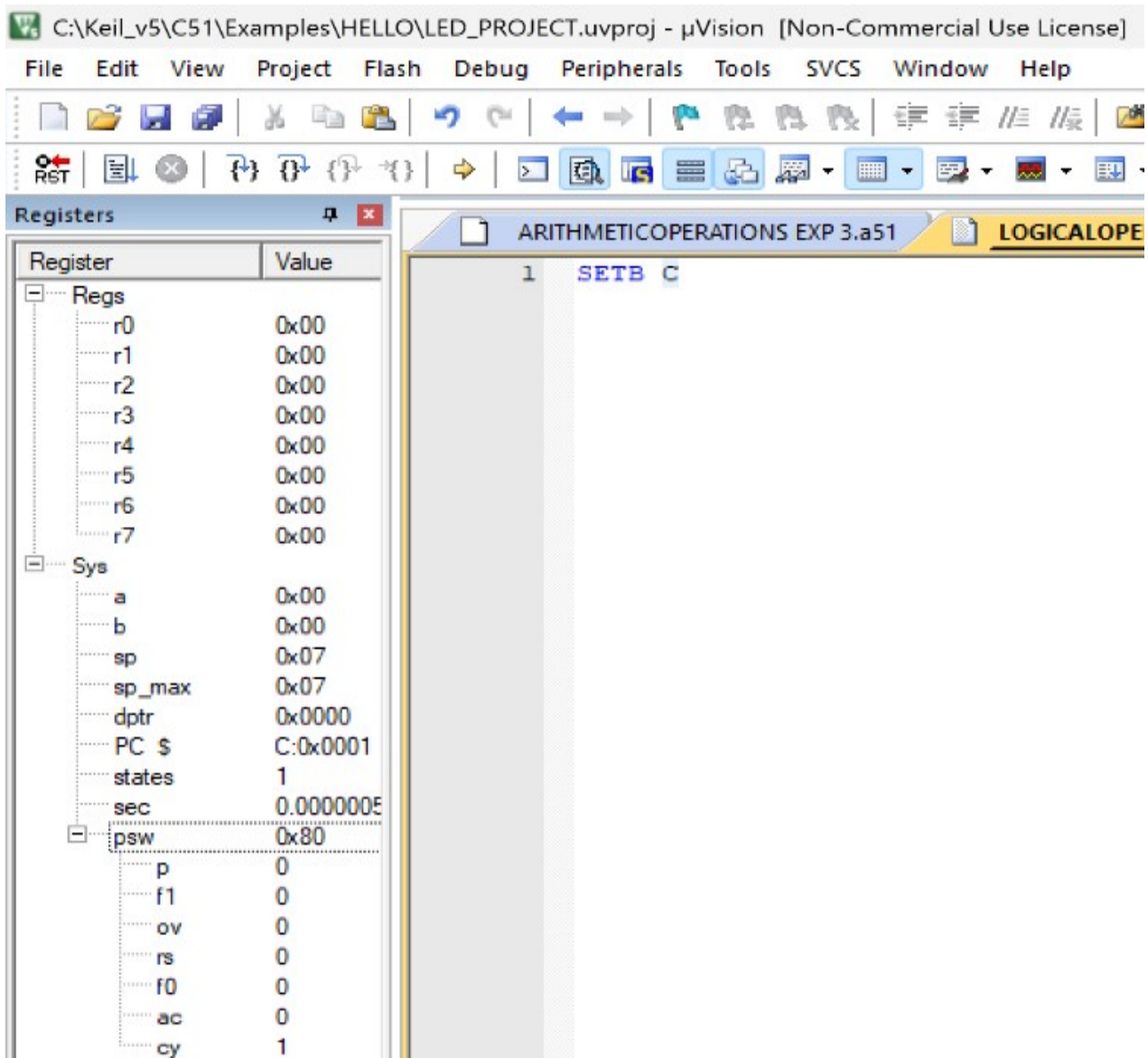
SETB C

Output:





5

**(ix): Clear Carry Flag****Objective:** To clear the Carry flag.**Input:**

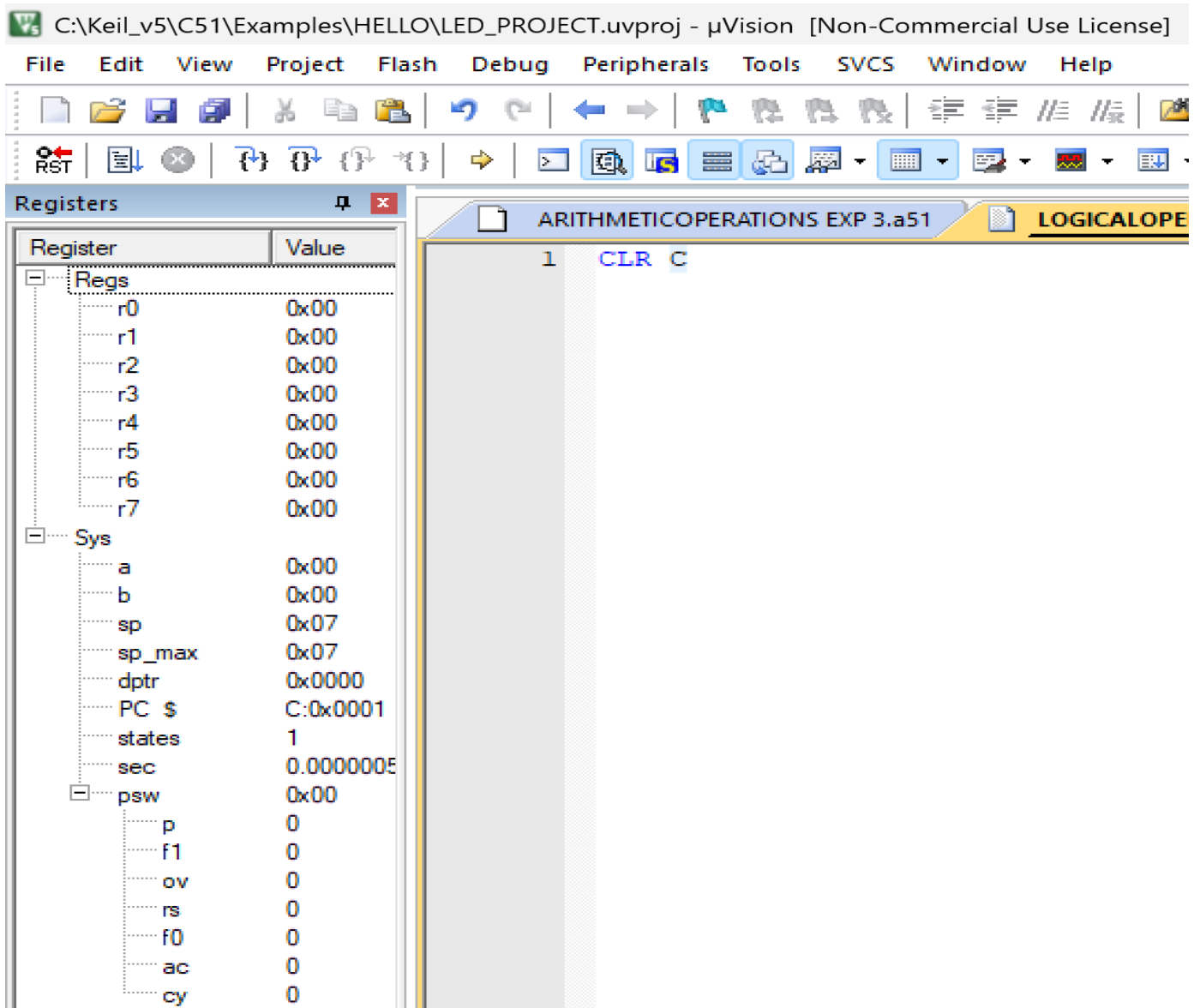
- Carry Flag = 1

Result:

- Carry Flag = 0

Instruction:

CLR C

Output:**RESULT:**

Thus, the arithmetic operations (Addition, Subtraction, Multiplication, and Division) and logical operations (AND, OR, XOR, Complement, Rotate Left, and Rotate Right) were successfully implemented using 8051 Assembly Language in Keil uVision IDE.

EXP NO.4	8051 ALP FOR DATA TRANSFER BETWEEN MEMORY BLOCKS
DATE: 07.08.26	

A) Transfer the Block of Data between two Memory Locations:

AIM:

To write an Assembly Language Program (ALP) to transfer a block of data from one internal memory location to another.

ALGORITHM:

1. Initialize the source memory address.
2. Initialize the destination memory address.
3. Load the counter with the number of bytes to be transferred.
4. Read one byte from the source location.
5. Store the byte in the destination location.
6. Increment both source and destination pointers.
7. Decrement the counter.
8. Repeat until the counter becomes zero.
9. Stop the program.

PROGRAM:

```

ORG 00H
MOV R0,#60H
MOV R1,#70H
MOV R2,#04H
L1: MOV A,@R0
50
MOV @R1,A
    INC R0
    INC R1
    DJNZ R2,L1
END

```

OUTPUT:

[illegible]

[illegible]

RESULT:

Thus, the block of data was successfully transferred from one internal memory location to another and verified successfully.

B) Exchange (Swap) the Block of Data Between Two Memory Locations

AIM:

To write an Assembly Language Program (ALP) to exchange (swap) the contents of a block of data between two internal memory locations.

ALGORITHM:

1. Initialize the starting address of Block 1.
2. Initialize the starting address of Block 2.
3. Load the counter with the number of bytes.
4. Read one byte from Block 1 into the Accumulator.
5. Store it temporarily in Register B.
6. Read one byte from Block 2 into the Accumulator.
7. Move the contents of Register B to Block 2.
8. Move the Accumulator contents to Block 1.
9. Increment both memory pointers.
10. Decrement the counter.
11. Repeat until all bytes are exchanged.
12. Stop the program.

PROGRAM:

```

ORG 00H
MOV R0,#50H
MOV R1,#80H
MOV R2,#04H
L1: MOV A,@R0
    MOV B,A
    MOV A,@R1
    MOV @R0,A
    MOV A,B
    MOV @R1,A
    INC R0
    INC R1
    DJNZ R2,L1
END

```

OUTPUT:

[illegible]

RESULT:

Thus, the contents of the two memory blocks were successfully exchanged and verified successfully.

EXP NO.5	LARGEST AND SMALLEST NUMBER IN AN ARRAY USING 8051 ALP
DATE: 18.09.26	

AIM:

To write and execute an 8051 Assembly Language program to find the largest and smallest numbers in an array and store the results in the specified memory location.

(a) LARGEST NUMBER IN AN ARRAY**ALGORITHM:**

1. Start the program.
2. Load the number of array elements into register R2.
3. Initialize R0 with the starting address of the array.
4. Move the first array element into accumulator A.
5. Initialize R1 with the address of the next element.
6. Compare the current largest value in A with the next array element.
7. If the next element is larger, move it into A.
8. Increment the array pointer.
9. Repeat the comparison until all elements are checked.
10. Store the largest value from A into memory location 70H.
11. Stop.

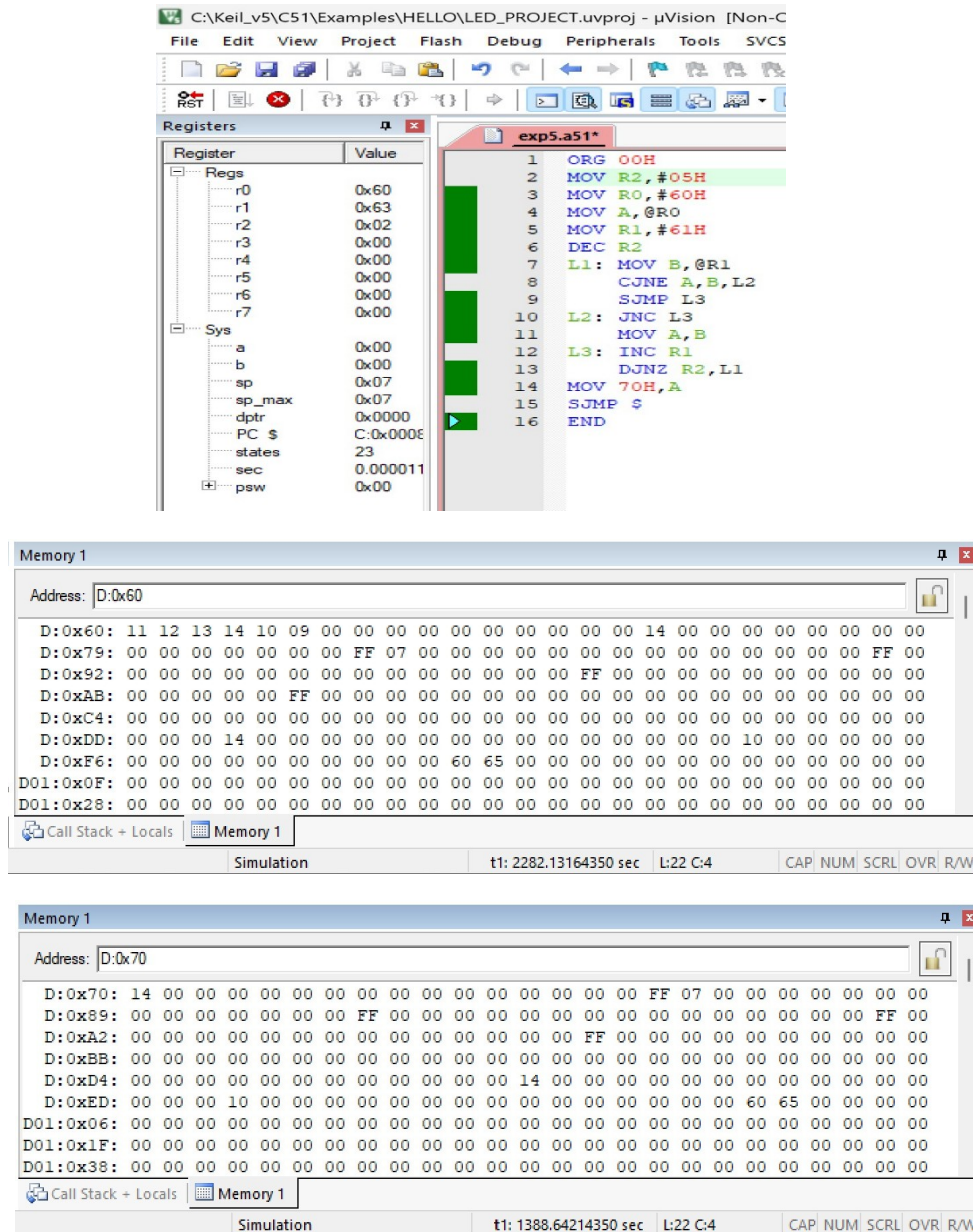
INSTRUCTION:

```

ORG 00H
MOV R2,50H
DEC R2
MOV R0,#60H
MOV A,@R0
MOV R1,#61H
L1: MOV 00H,@R1
CJNE A,00H,L2
L2: JNC L3
MOV A,@R1
L3: INC R1
DJNZ R2,L1
MOV 70H,A
END

```

OUTPUT:



(b) SMALLEST NUMBER IN AN ARRAY

Algorithm

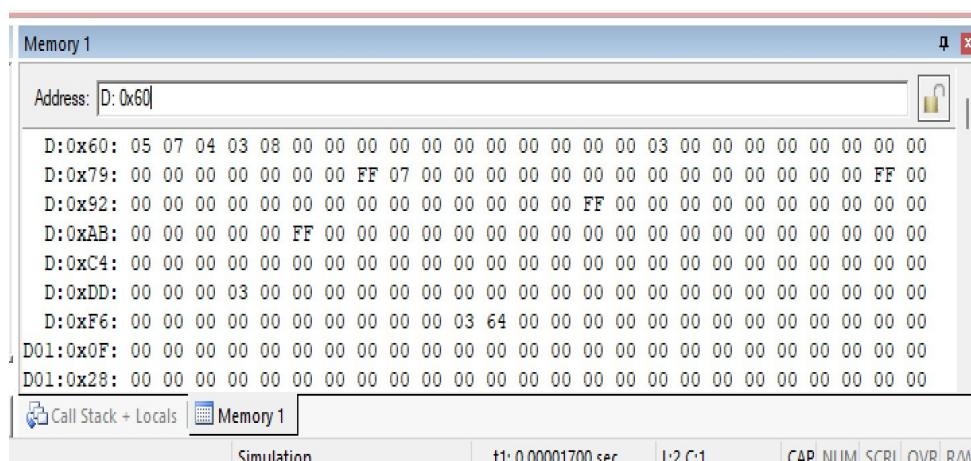
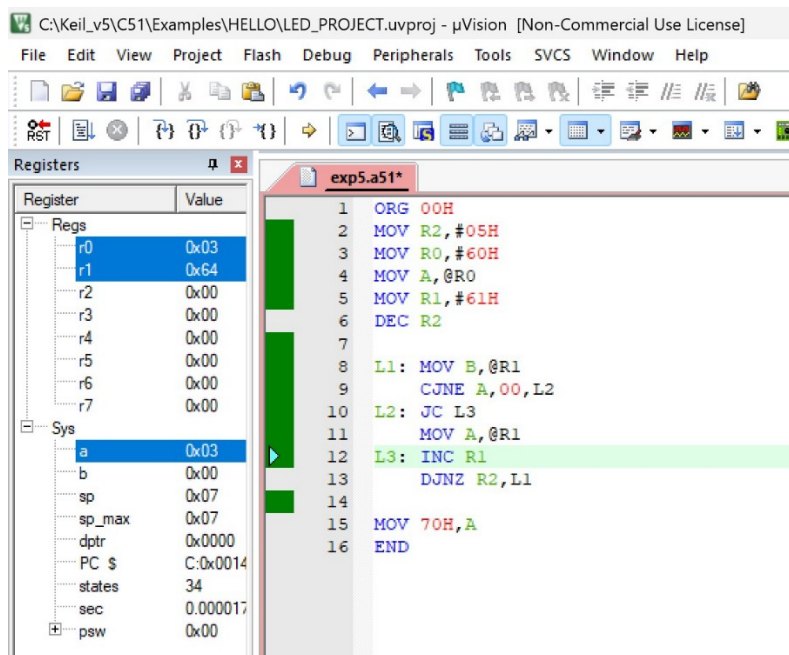
1. Start the program.
2. Load the number of array elements into register R2.
3. Initialize R0 with the starting address of the array.
4. Move the first array element into accumulator A.
5. Initialize R1 with the address of the next element.
6. Compare the current largest value in A with the next array element.
7. If the next element is larger, move it into A.
8. Increment the array pointer.
9. Repeat the comparison until all elements are checked.
10. Store the largest value from A into memory location 70H.
11. Stop.

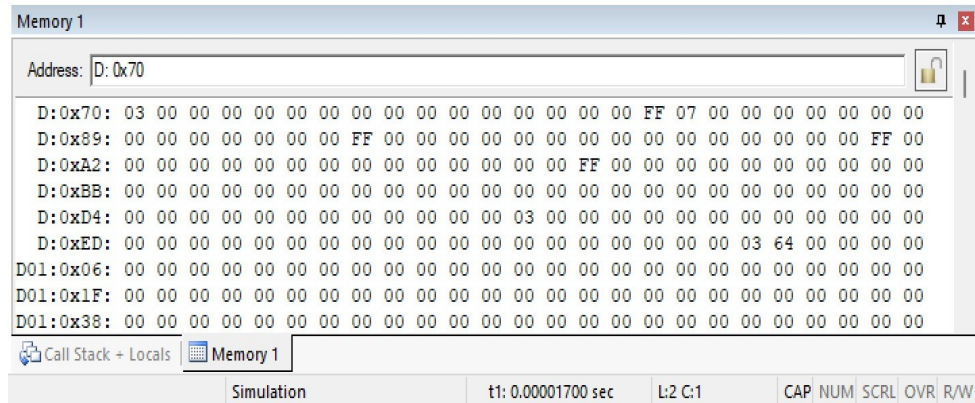
INSTRUCTION:

```

ORG 00H
MOV R2,50H
DEC R2
MOV R0,#60H
MOV A,@R0
MOV R1,#61H
L1: MOV 00H,@R1
CJNE A,00H,L2
L2: JNC L3
MOV A,@R1
L3: INC R1
DJNZ R2,L1
MOV 70H,A
END

```

OUTPUT:



RESULT:

Thus, the 8051 Assembly Language programs for finding the largest and smallest numbers in an array were executed successfully.

EXP NO.6	8051 C PROGRAMMING FOR TIMER
DATE: 18.09.26	

(a) 8051 TIMER MODE 1 WITH DELAY**AIM:**

To write and execute an 8051 C program using Timer 0 in Mode 1 to generate a delay and enable/disable a port pin.

ALGORITHM:

1. Start the program.
2. Define the port pin P1.0 as the test pin.
3. Configure Timer 0 in Mode 1 (16-bit timer mode).
4. Load the initial values into TH0 and TL0.
5. Start Timer 0.
6. Wait until the Timer 0 overflow flag TF0 is set.
7. Stop the timer and clear TF0.
8. Toggle the P1.0 pin.
9. Repeat the process continuously.

PROGRAM:

```
#include <reg51.h>
sbit test = P1^0;
void timer_delay()
{
    TH0 = 0xFC;
    TL0 = 0x74;
    TR0 = 1;
    while(TF0 == 0);
    TR0 = 0;
    TF0 = 0;
}
void main()
{
    TMOD = 0x01;
    while(1)
    {
        test = ~test;
        timer_delay();
    }
}
```

OUTPUT:

The screenshot shows the Keil uVision IDE with the Registers window open on the left and the source code for exp6.c in the main editor. The Registers window shows the status of various registers, with the stack pointer (sp) highlighted at 0x07. The source code defines a timer_delay function and a main function that toggles a pin (P1^0) using a timer interrupt.

```

1  #include <reg51.h>
2
3  sbit test = P1^0;
4
5  void timer_delay()
6  {
7      TH0 = 0xFC;
8      TL0 = 0x74;
9
10     TR0 = 1;
11     while(TF0 == 0);
12
13     TR0 = 0;
14     TF0 = 0;
15 }
16
17 void main()
18 {
19     TMOD = 0x01;
20
21     while(1)
22     {
23         test = ~test;
24         timer_delay();
25     }
26 }

```

Parallel Port 1

Port 1

P1: 0xFE 7 Bits 0

Pins: 0xFE

Parallel Port 1

Port 1

P1: 0xFF 7 Bits 0

Pins: 0xFF

Timer/Counter 0

Mode: 1: 16 Bit Timer/Counter

Timer

TCON: 0x10 TMOD: 0x01

TH0: 0xFC TL0: 0xA7

☒ T0 Pin ☐ TF0

Control

Status: Run

☒ TR0 ☐ GATE ☒ INT0#

Timer/Counter 0

Mode: 1: 16 Bit Timer/Counter

Timer

TCON: 0x10 TMOD: 0x01

TH0: 0xFD TL0: 0x0D

☒ T0 Pin ☐ TF0

Control

Status: Run

☒ TR0 ☐ GATE ☒ INT0#

Timer/Counter 0

Mode: 1: 16 Bit Timer/Counter

Timer

TCON: 0x10 TMOD: 0x01

TH0: 0xFE TL0: 0x09

☒ T0 Pin ☐ TF0

Control

Status: Run

☒ TR0 ☐ GATE ☒ INT0#

Timer/Counter 0

Mode: 1: 16 Bit Timer/Counter

Timer

TCON: 0x10 TMOD: 0x01

TH0: 0xFF TL0: 0x17

☒ T0 Pin ☐ TF0

Control

Status: Run

☒ TR0 ☐ GATE ☒ INT0#

(b) TIMER-BASED INTERRUPT TO TOGGLE OUTPUT PIN**AIM**

To write and execute an 8051 C program using a timer-based interrupt to toggle an output pin at a fixed time interval.

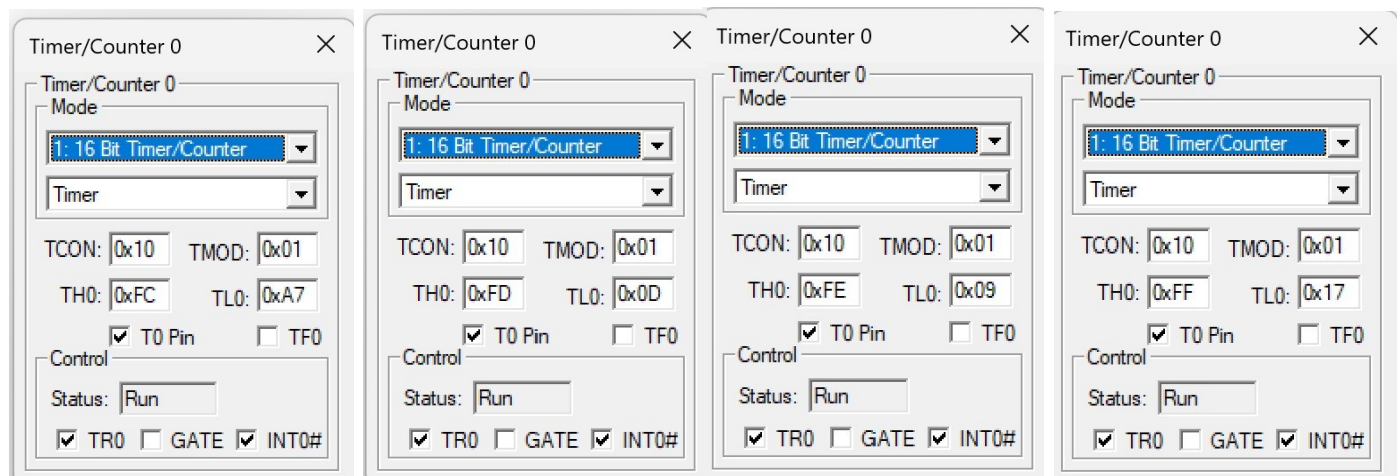
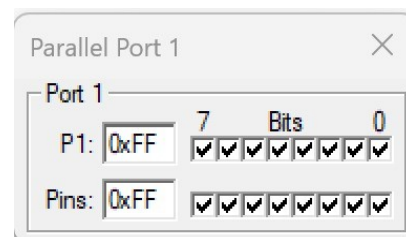
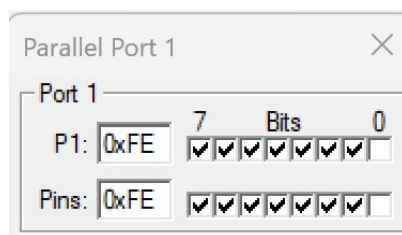
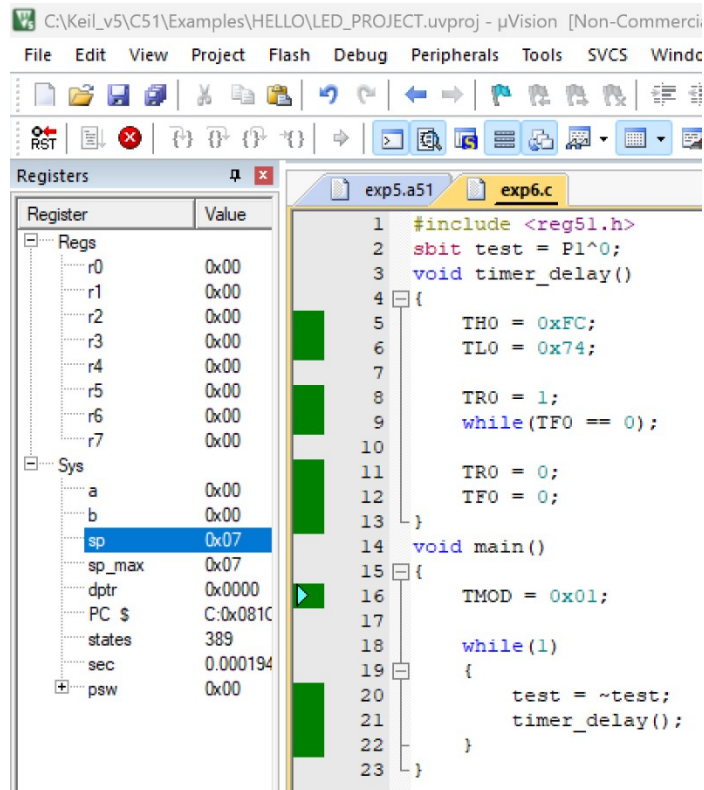
ALGORITHM:

1. Start the program.
2. Define P1.0 as the output/test pin.
3. Configure Timer 0 in Mode 1.
4. Load TH0 and TL0 with the required initial values.
5. Enable the global interrupt.
6. Enable Timer 0 interrupt.
7. Start Timer 0.
8. When Timer 0 overflows, execute the interrupt service routine.
9. Toggle P1.0.
10. Reload TH0 and TL0.
11. Repeat the process continuously.

PROGRAM:

```
#include <reg51.h>
sbit test = P1^0;
void Timer_init(void)
{
    TMOD = 0x01;
    TH0 = 0x4C;
    TL0 = 0x00;
    TR0 = 1;
}
void Timer0_ISR(void) interrupt 1
{
    test = ~test;
    TH0 = 0x4C;
    TL0 = 0x00;
}
int main(void)
{
    EA = 1;
    ET0 = 1;
    Timer_init();
    while(1);
}
```

OUTPUT:



(c) UART PROGRAMMING USING 8051

AIM:

To write and execute an 8051 C program for Universal Asynchronous Receiver Transmitter (UART) to transmit a character serially.

ALGORITHM:

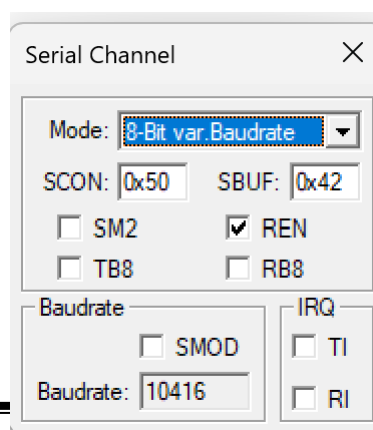
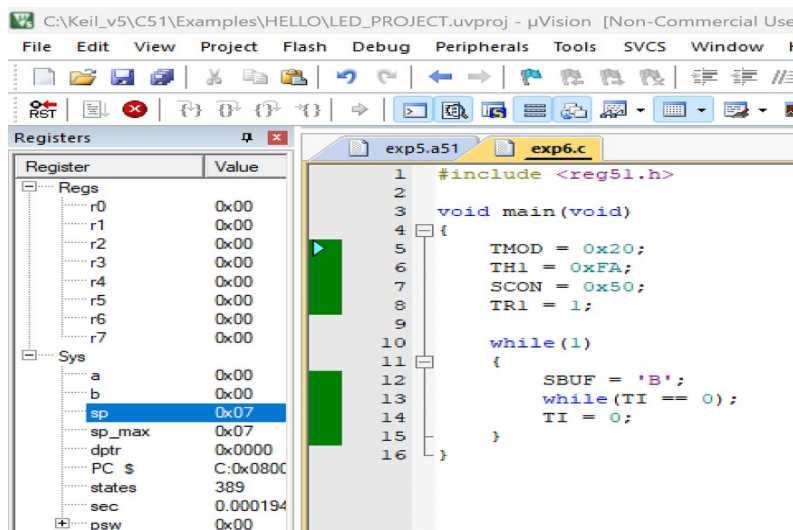
1. Start the program.

2. Configure Timer 1 in Mode 2 for baud-rate generation.
3. Load the required value into TH1.
4. Configure the serial control register SCON.
5. Start Timer 1.
6. Load the character to be transmitted into SBUF.
7. Wait until the transmission interrupt flag TI is set.
8. Clear the TI flag.
9. Repeat the transmission continuously.

PROGRAM

```
#include <reg51.h>
void main(void)
{
    TMOD = 0x20;
    TH1 = 0xFA;
    SCON = 0x50;
    TR1 = 1;
    while(1)
    {
        SBUF = 'B';
        while(TI == 0);
        TI = 0;
    }
}
```

OUTPUT:



EXP NO.7	8051 C PROGRAMMING FOR PORT HANDLING OPERATIONS
DATE: 25.09.26	

RESULT:

Thus, the 8051 C programs were successfully implemented for timer delay and port toggling, timer-based interrupt, and UART serial data transmission.

AIM:

To write and execute 8051 C programs for ASCII number display through a port, I/O port programming with delay, and toggling between two or more I/O ports.

(a) To Display ASCII Number in a Port

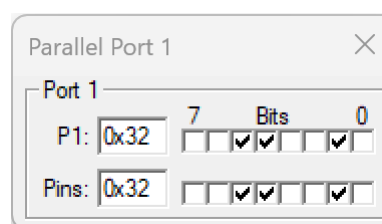
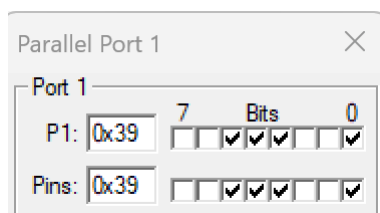
ALGORITHM:

1. Start the program.
2. Include the REG51.H header file.
3. Declare variables x and y.
4. Store the hexadecimal number 29H in mb.
5. Extract the lower nibble using mb & 0x0F.
6. Add 30H to convert the lower nibble into its ASCII value.
7. Display the result through Port 1.
8. Extract the upper nibble using mb & 0xF0.
9. Shift the upper nibble right by 4 positions.
10. Add 30H to obtain its ASCII value.
11. Display the result through Port 1.
12. Repeat continuously.

PROGRAM:

```
#include<reg51.h>
void main(void){
while(1){
unsigned char x,y;
unsigned char mb = 0x29 ;
x = mb &0x0F;
P1 = x|0x30;
y = mb & 0xF0;
y = y >>4;
P1 = y|0x30;}}
```

OUTPUT:



(B) I/O Port Programming with Delay

ALGORITHM:

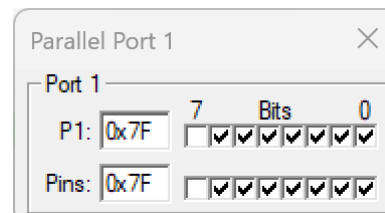
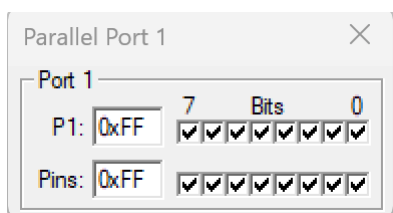
1. Start the program.
2. Include the REG51.H header file.
3. Define P1.7 as mybit.
4. Create a delay function.
5. Set mybit to 0.

6. Generate a delay.
7. Set mybit to 1.
8. Generate another delay.
9. Repeat the process continuously.
10. Observe the toggling of Port 1 bit 7.

PROGRAM:

```
#include <reg51.h>
sbit mybit = P1^7;
void delay(unsigned int a);
void main(void)
{
    while(1)
    {
        mybit = 0;
        delay(20);
        mybit = 1;
        delay(20);
    }
}

void delay(unsigned int a)
{
    unsigned int i, j;
    for(i = 0; i < a; i++)
        for(j = 0; j < 50; j++);
}
```

OUTPUT:


(C) 8051 I/O Port Programming to Toggle Between Two or More Ports**ALGORITHM:**

1. Start the program.
2. Include the REG51.H header file.
3. Initialize Port 0, Port 1, Port 2 and Port 3 with 55H.
4. Complement the value of Port 0.
5. Complement the value of Port 1.
6. Complement the value of Port 2.
7. Complement the value of Port 3.
8. Generate a small delay.
9. Repeat the process continuously.
10. Observe the ports changing between 55H and AAH.

PROGRAM:

```
#include <reg51.h>
void MsDelay(unsigned int);
void main()
{
    P0 = P1 = P2 = P3 = 0x33;
    while(1)
    {
        P0 = ~P0;
        P1 = ~P1;
        P2 = ~P2;
        P3 = ~P3;
        MsDelay(1);
    }
}
void MsDelay(unsigned int itime)
{
    unsigned int i, j;
    for(i = 0; i < itime; i++)
        for(j = 0; j < 3; j++);
}
```

OUTPUT:

```

exp7.c*
1  #include <reg51.h>
2  void MsDelay(unsigned int);
3  void main()
4  {
5      P0 = P1 = P2 = P3 = 0x33;
6      while(1)
7      {
8          P0 = ~P0;
9          P1 = ~P1;
10         P2 = ~P2;
11         P3 = ~P3;
12
13         MsDelay(1);
14     }
15 }
16 void MsDelay(unsigned int itime)
17 {
18     unsigned int i, j;
19     for(i = 0; i < itime; i++)
20         for(j = 0; j < 3; j++);
21 }

```

Parallel Port 1

Port 1

P1: 0xCC 7 Bits 0

Pins: 0xCC

7	6	5	4	3	2	1	0
✓	✓	✓	✓	✓	✓	✓	✓

Parallel Port 1

Port 1

P1: 0x33 7 Bits 0

Pins: 0x33

7	6	5	4	3	2	1	0
✓	✓	✓	✓	✓	✓	✓	✓

RESULT:

Thus, the 8051 C programs for ASCII number display, I/O port programming with delay, and toggling between multiple I/O ports were executed successfully, and the expected port outputs were obtained.

EXP NO.8	8051 C PROGRAMMING FOR INTERFACING LED
DATE:	

A. 8051 Programming to Blink the LED with Delay**AIM:**

To demonstrate the interfacing of LEDs with the 8051 microcontroller and to control/blink the LEDs with a time delay.

PROCEDURE:

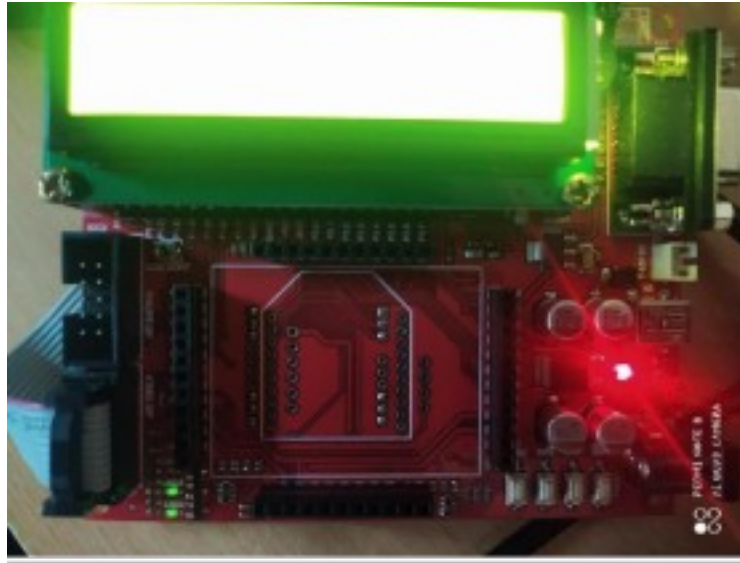
1. Create a new project.
2. Select the microcontroller Atmel → AT89C52.
3. Enter the source code.
4. Save the program.
5. Add the HEX file to the output.
6. Compile the program using Project → Build Target / F7.
7. The HEX file will be generated in the project folder.

PROGRAM:

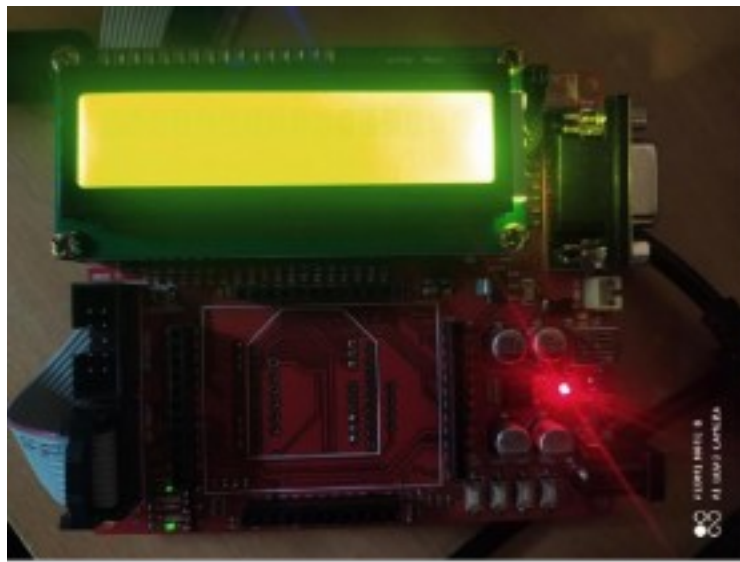
```
#include <REGX52.H>
#define LED1 P1_5
#define LED2 P1_4
#define LED3 P1_3
void delay(unsigned int count);
void main()
{
    while(1)
    {
        LED1 = 0;
        LED2 = 1;
        LED3 = 0;
        delay(50);
        LED1 = 1;
        LED2 = 0;
        LED3 = 0;
        delay(50);
    }
}
void delay(unsigned int count)
{
    int x, y;
    for(x = 0; x < count; x++)
        for(y = 0; y < 1275; y++);
}
```

OUTPUT:

BEFORE EXECUTION:



AFTER EXECUTION:



B. LED with Switch Control

AIM:

To control the LEDs using switches connected to the 8051 microcontroller and produce a beep when a switch is pressed.

ALGORITHM:

1. Start the program.
2. Include the required header file and define the ports for LEDs, switches and beep.
3. Run an infinite while loop for continuous switch monitoring.
4. Check whether a particular switch is pressed.
5. When a switch is pressed, produce a beep for a short delay and switch ON the corresponding LED.
- 6.

PROGRAM:

```

#include <REGX52.H>
#define EN P1_0
#define RS P1_2
#define SW1 P2_0
#define SW2 P2_1
#define SW3 P2_2
#define beep P1_6
#define LED0 P1_5
#define LED1 P1_4
#define LED2 P1_3
void delay(unsigned int count);
void main()
{
    while(1)
    {
        if(SW1 == 0)
        {
            while(!SW1);
            beep = 0;
            delay(25);
            beep = 1;
            LED0 = 0;
            LED1 = 1;
            LED2 = 1;
        }
        if(SW2 == 0)
        {
            while(!SW2);
            beep = 0;
            delay(25);
            beep = 1;
            LED0 = 1;
            LED1 = 0;
            LED2 = 1;
        }
        if(SW3 == 0)
        {
            while(!SW3);
            beep = 0;
            delay(25);
            beep = 1;
            LED2 = 0;
            LED1 = 1;
            LED0 = 1;
        }
    }
}

```

```
    }  
  }  
}  
void delay(unsigned int count)  
{  
  int x, y;  
  for(x = 0; x < count; x++)  
    for(y = 0; y < 1275; y++);  
}
```

OUTPUT:



LED1



LED2



LED3

RESULT

Thus, the 8051 C programming for LED interfacing was executed and verified successfully.

EXP NO.9	8051 C PROGRAMMING FOR INTERFACING LCD
DATE:	

A. 8051 C programming to interface with LCD and display the given input**AIM:**

To interface an LCD with the 8051 microcontroller and display the given text, as well as display different messages on the LCD based on the switch pressed.

ALGORITHM:

1. Start the program.
2. Include the required header file and define the LCD data port, EN, RS and RW pins.
3. Initialize the LCD using the required commands.
4. Send the required text to the LCD.
5. Run the program continuously.
6. Stop the program.

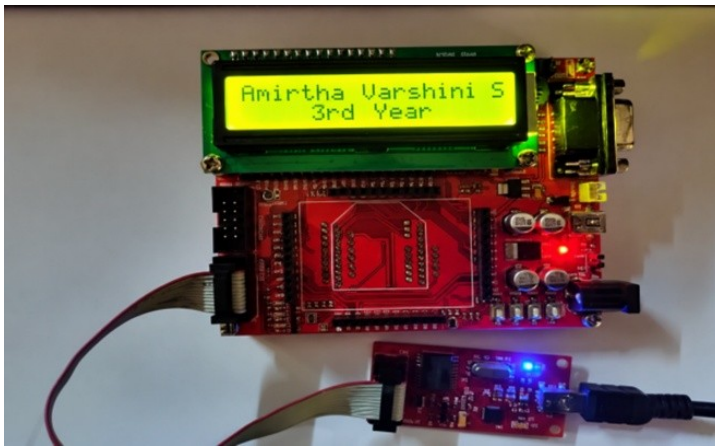
HEXCODE COMMANDS:

30 1 Line, 5x7 matrix
 38 2 lines and 5x7 matrix
 01 Clear display screen
 06 Increment cursor (shift cursor to right)
 0C Display on, cursor off
 80 Force cursor to beginning (1st line)
 C0 Force cursor to beginning (2nd line)

PROGRAM:

```
#include <REGX52.H>
#define LCD_PORT P0
#define EN P1_0
#define RS P1_2
#define RW P1_1
void lcd_initial(void);
void lcd_command(char);
void lcd_data(char);
void lcd_display(unsigned char, const char *);
void delay(unsigned int);
void main()
{
    lcd_initial();
    lcd_display(0x80, "Amirtha Varshini S");
    lcd_display(0xC0, "3rd Year");
    while(1);
}
```

```
}  
void lcd_initial(void)  
{  
  
    lcd_command(0x30);  
    lcd_command(0x30);  
    lcd_command(0x30);  
    lcd_command(0x38);  
    lcd_command(0x01);  
    lcd_command(0x06);  
    lcd_command(0x0C);  
    lcd_command(0x80);  
}  
void lcd_command(char value)  
{  
    RW = 0;  
    RS = 0;  
    EN = 1;  
    LCD_PORT = value;  
    delay(1);  
    EN = 0;  
}  
void lcd_data(char value)  
{  
    RW = 0;  
    RS = 1;  
    EN = 1;  
    LCD_PORT = value;  
    delay(1);  
    EN = 0;  
}  
void lcd_display(unsigned char locn, const char *dat)  
{  
    lcd_command(locn);  
    while(*dat != '\0')  
    {  
        lcd_data(*dat);  
        dat++;  
    }  
}  
void delay(unsigned int count)  
{  
    int x, y;  
    for(x = 0; x < count; x++)  
        for(y = 0; y < 1275; y++);  
}
```

OUTPUT:**B. LCD WITH SWITCH CONTROL****AIM:**

To interface an LCD with the 8051 microcontroller and display different messages depending on which switch is pressed.

ALGORITHM:

1. Start the program.
2. Include the required header file and define LCD, switch, EN, RS, RW and buzzer pins.
3. Initialize the LCD.
4. Display “Press Any Switch” initially.
5. Continuously check the switches.
6. If SW1 is pressed, display “Pressed Switch 1”.
7. If SW2 is pressed, display “Pressed Switch 2”.
8. If SW3 is pressed, display “Pressed Switch 3”.
9. Stop the program. MCP record from Exp 7

PROGRAM:

```
#include <REGX52.H>
#define LCD_PORT P0
#define EN P1_0
#define RS P1_2
#define RW P1_1
#define SW1 P2_0
#define SW2 P2_1
#define SW3 P2_2
#define beep P1_6
void Lcd_Initial(void);
void lcd_command(char);
void lcd_data(char);
void lcd_display(unsigned char, const char *);
```

```
void delay(unsigned int);
void main()
{
    P2 = 0x07;
    Lcd_Initial();
    lcd_display(0x80, "Press Any Switch");
    delay(100);

    while(1)
    {
        if(SW1 == 0)
        {
            while(!SW1);
            beep = 0;
            delay(25);
            beep = 1;
            lcd_display(0x80, "Pressed Switch 1");
        }
        if(SW2 == 0)
        {
            while(!SW2);
            beep = 0;
            delay(25);
            beep = 1;
            lcd_display(0x80, "Pressed Switch 2");
        }
        if(SW3 == 0)
        {
            while(!SW3);
            beep = 0;
            delay(25);
            beep = 1;
            lcd_display(0x80, "Pressed Switch 3");
        }
    }
}

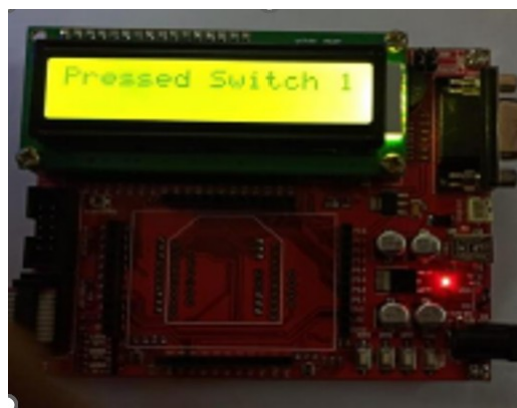
void Lcd_Initial(void)
{
    lcd_command(0x30);
    lcd_command(0x30);
    lcd_command(0x30);
    lcd_command(0x38);
    lcd_command(0x01);
    lcd_command(0x06);
    lcd_command(0x0C);
}
```

```
void lcd_command(char value)
{
    RW = 0;
    RS = 0;
    EN = 1;
    LCD_PORT = value;
    delay(1);
    EN = 0;
}

void lcd_data(char value)
{
    RW = 0;
    RS = 1;
    EN = 1;
    LCD_PORT = value;
    delay(1);
    EN = 0;
}

void lcd_display(unsigned char locn, const char *dat)
{
    lcd_command(locn);
    while(*dat != '\0')
    {
        lcd_data(*dat);
        dat++;
    }
}

void delay(unsigned int count)
{
    int x, y;
    for(x = 0; x < count; x++)
        for(y = 0; y < 1275; y++);
}
```

OUTPUT:



\

RESULT:

Thus, the 8051 C programming for LCD interfacing and LCD with switch control was executed and verified successfully.

EXP NO.10	ARM7 PROGRAMMING FOR LED AND LCD INTERFACE
DATE:	

A. Interfacing ARM7 with LED using LPC2148.**AIM:**

To demonstrate the interfacing of LED and LCD with the ARM7 LPC2148 processor and to control the LED and display the given text on the LCD.

PROCEDURE:

1. Create a new project.
2. Select ARM → ARM7.
3. Enter the source code.
4. Save the program.
5. Add the source code to the target.
6. Compile and build the target.
7. Generate the HEX file.
8. Add the HEX file to the LPC2000 Flash Utility.
9. Transfer the HEX file to the ARM processor.
10. End the program.

PROGRAM:

```
#include <lpc21xx.h>

void delay(unsigned long val);

int main()
{
    IO1DIR = 0x00FF0000;

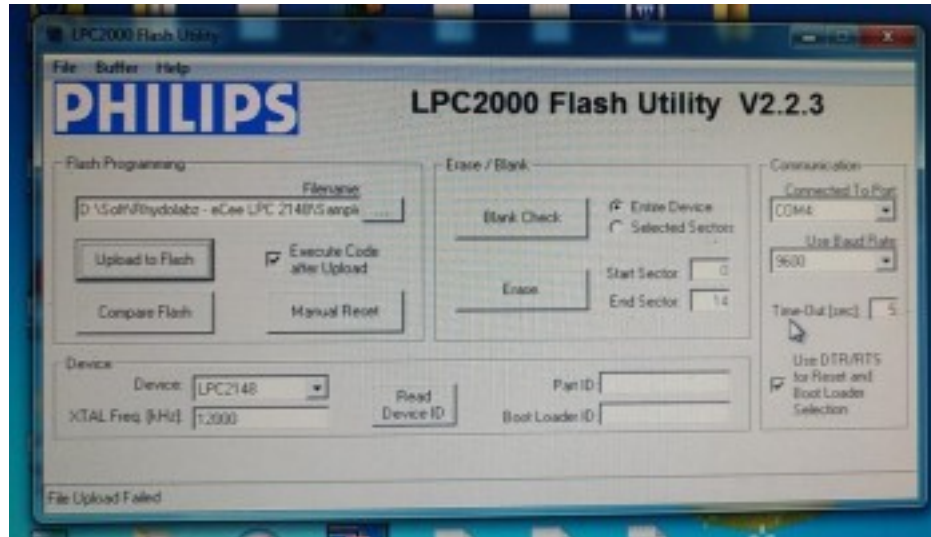
    while(1)
    {
        IO1SET = 0x00FF0000;
        delay(1000000);

        IO1CLR = 0x00FF0000;
        delay(1000000);
    }
}

void delay(unsigned long val)
{
    while(val > 0)
```

```
val--;  
}
```

PHILIPS LPC2000 FLASH UTILITY



OUTPUT:



B. ARM7 Programming for LCD Interface**AIM:**

To interface an LCD with the ARM7 LPC2148 processor and display the given text.

PROCEDURE:

1. Create a new project.
2. Select ARM → ARM7.
3. Enter the source code.
4. Save the program.
5. Add the source code to the target.
6. Compile and build the target.
7. Generate the HEX file.
8. Add the HEX file to the LPC2000 Flash Utility.
9. Transfer the HEX file to the ARM processor.
10. End the program. MCP record from Exp 7

PROGRAM:

```
#include <LPC21xx.h>
#include "lcd4bit.h"
void Delay(unsigned long b)
{
    while(--b != 0);
}
void write_command(int cmd)
{
    IO1CLR |= 0x00f00000;
    IO1CLR |= 0x00040000;
    IO1CLR |= 0x00020000;
    IO1SET |= 0x00f00000 & cmd;
    IO1SET |= 0x00080000;
    Delay(30000);
    IO1CLR |= 0x00080000;
}
void write_data(int dat)
{
    IO1CLR |= 0x00f00000;
    IO1CLR |= 0x00040000;
    IO1SET |= 0x00020000;
    IO1SET |= 0x00f00000 & dat;
    IO1SET |= 0x00080000;
    Delay(30000);
    IO1CLR |= 0x00080000;
}
void lcd_data(char dat)
{

```

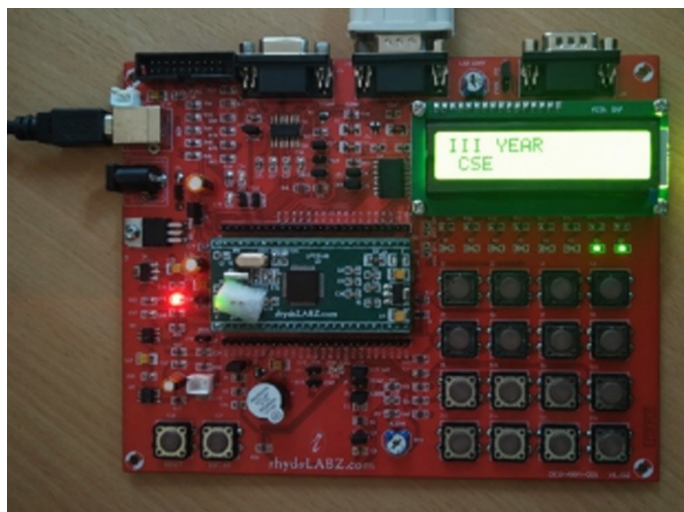
```
    write_data(dat << 16);
    write_data(dat << 20);
}
void lcd_command(char cmd)
{
    write_command(cmd << 16);
    write_command(cmd << 20);
}
void printlcd(char *CPtr)
{
    while(*CPtr != '\0')
    {
        lcd_data(*CPtr);
        CPtr++;
        Delay(20000);
    }
}
void init_lcd(void)
{
    IO1DIR |= 0x00FE0000;
    Delay(200000);
    write_command(0x30 << 16);
    Delay(100000);
    write_command(0x30 << 16);
    Delay(100000);
    write_command(0x30 << 16);
    Delay(100000);
    write_command(0x20 << 16);
    lcd_command(0x01);
    lcd_command(0x06);
    lcd_command(0x0c);
    lcd_command(0x80);
}

int main(void)
{
    init_lcd();
    printlcd("eCee LPC2148 DLB");
    lcd_command(0xC0);
    printlcd(" rhydolabz.com ");
    while(1);
}
```

PHILIPS LPC2000 FLASH UTILITY :



OUTPUT:



RESULT:

Thus, the ARM7 LED and LCD interfacing programs were executed and verified successfully.

EXP NO.11	ARM7 PROGRAMMING FOR BUZZER INTERFACE
DATE:	

AIM:

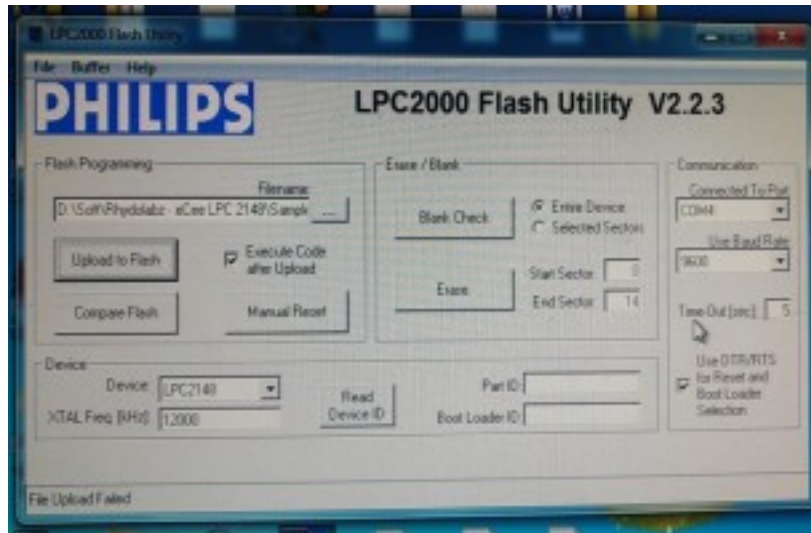
To demonstrate the process of interfacing a buzzer with the ARM7 processor.

PROCEDURE:

1. Create a new project.
2. Select ARM → ARM7.
3. Enter the source code.
4. Save the program.
5. Add the source code to the target.
6. Compile and build the target.
7. Generate the HEX file.
8. Add the HEX file to the LPC2000 Flash Utility.
9. Transfer the HEX file to the ARM processor.
10. End the program.

PROGRAM:

```
#include <lpc21xx.h>
void ext_interrupt(void) irq
{
    EXTINT = 0X02;
    IO0PIN = (IO0PIN ^ 0X00010080) & 0X00010080;
    VICVectAddr = 0x00000000;
}
void init_ext(void)
{
    PINSEL0 |= 0X20000000;
    EXTMODE = 0X02;
    EXTPOLAR = 0X02;
    VICVectAddr0 = (unsigned int)ext_interrupt;
    VICVectCntl0 = 0x0000002F;
    VICIntEnable = 0x00008000;
}
int main(void)
{
    IO0DIR = 0X00010080;
    init_ext();
    while(1);
}
```



OUTPUT:



RESULT:

Thus, the ARM7 buzzer interfacing program was executed and verified successfully.

EXP NO.12	ARM7 PROGRAMMING FOR TIMER CONTROLLING OPERATION
DATE:	

AIM:

To demonstrate the process of LED blinking using a Timer with the ARM7 processor.

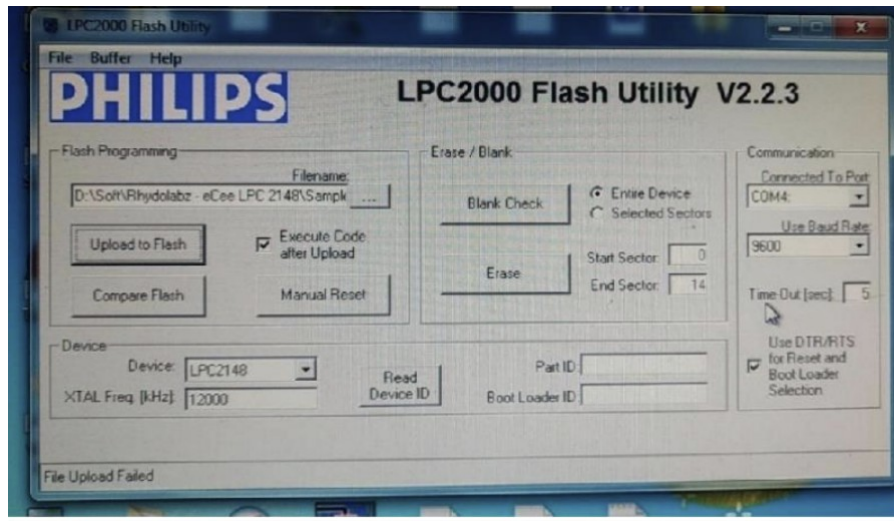
PROCEDURE:

1. Create a new project.
2. Select ARM → ARM7.
3. Enter the source code.
4. Save the program.
5. Add the source code to the target.
6. Compile and build the target.
7. Generate the HEX file.
8. Add the HEX file to the LPC2000 Flash Utility.
9. Transfer the HEX file to the ARM processor.
10. End the program.

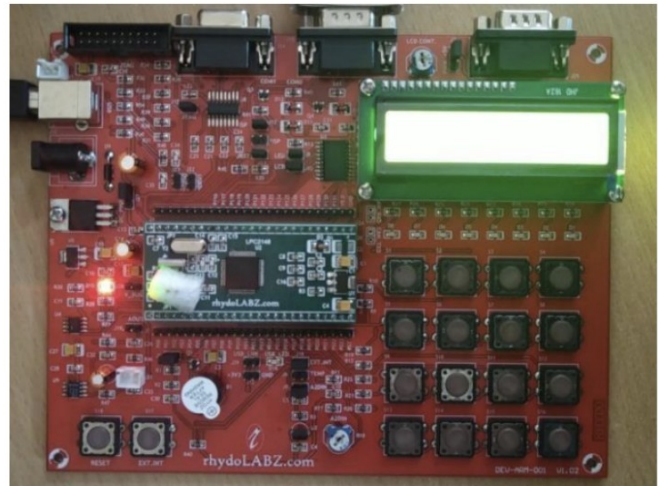
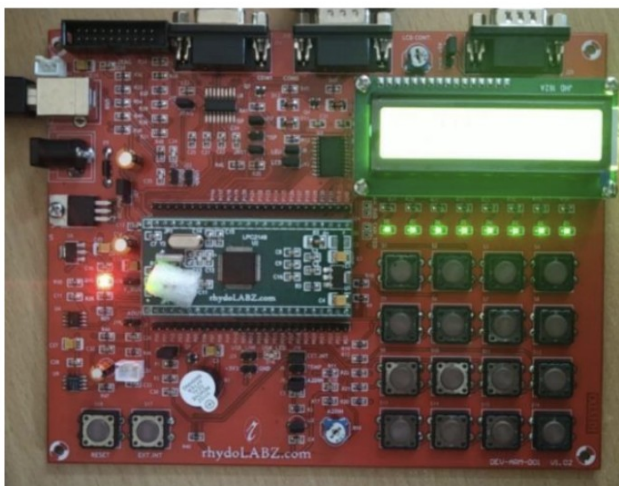
PROGRAM:

```
#include <lpc21xx.h>
void ext_interrupt(void) irq
{
    EXTINT = 0X02;
    IO0PIN = (IO0PIN ^ 0X00010080) & 0X00010080;
    VICVectAddr = 0x00000000;
}
void init_ext(void)
{
    PINSEL0 |= 0X20000000;
    EXTMODE = 0X02;
    EXTPOLAR = 0X02;
    VICVectAddr0 = (unsigned int)ext_interrupt;
    VICVectCntl0 = 0x0000002F;
    VICIntEnable = 0x00008000;
}
int main(void)
{
    IO0DIR = 0X00010080;
    init_ext();
    while(1);
}
```

PHILIPS LPC2000 FLASH UTILITY:



OUTPUT:



RESULT:

Thus, the ARM7 buzzer interfacing program was executed and verified successfully.